NM6008 • MODULE 6 • GF 22nm FDSOI

Module 6 后端实操
全流程 SOP & Debug 手册

Cadence Innovus 20.12 • Matrix Multiplier • 2026 实验版

按老师原版 Manual 的 15 个步骤执行；插入完整 GUI 截图、真实报错、最终签核检查点、恢复流程和工具命令对照。

面向考试 / 实验室 CentOS 7 / 独立复现实操

基础来源：Module 6 Laboratory Manual（22 页）；老师课堂更新照片；已提供的实验日志与截图（2026-10-25 至 2026-10-28）。

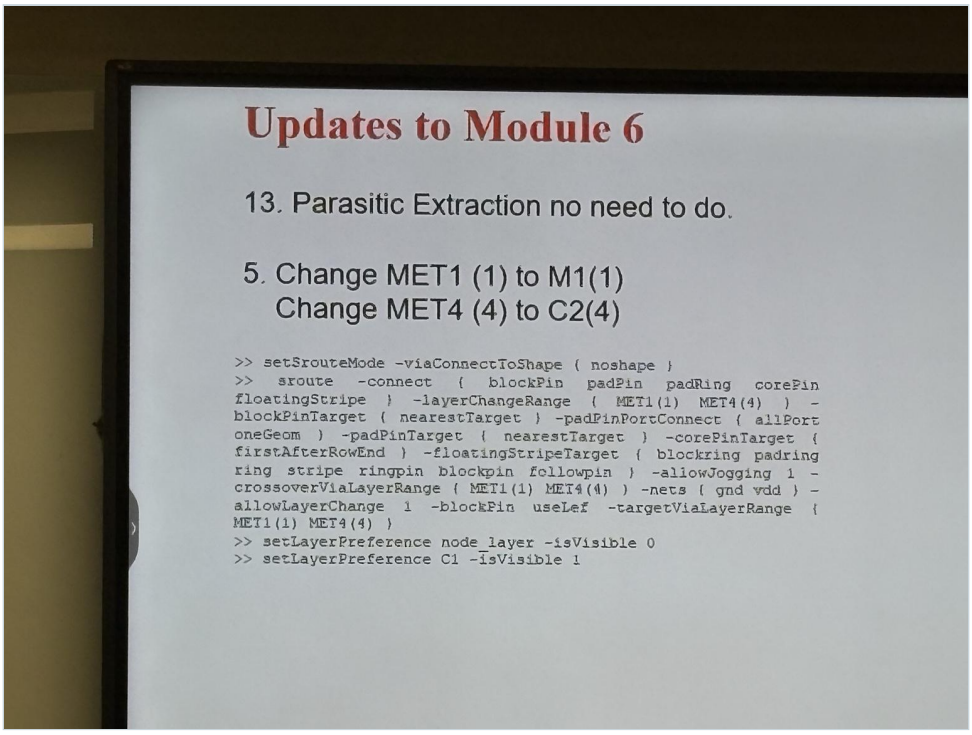
重要：这是复现实操手册，不保证特定机器、PDK、许可证和设计均无报错。遇到与本文不一致的报错时必须停止当前阶段并检查。

阅读导航 / 开始之前

本 PDF 的**前半部分为可全文搜索的中文 SOP 与故障索引**；最后附老师原版 22 页 Manual（全文和原 GUI 图），需要逐字对照时直接翻到附录。

章节	快速定位	说明
A	考试前 5 分钟检查	环境、shell、文件、禁止事项
B	原版 15 Steps 的逐步 SOP	每步：操作、命令、预期、异常、手册页
C	实战 Debug 词典	命令找不到、Hold、DRC、RC/GLIBC、Killed 等
D	复盘 / 收口 / 最终交付	证据表、报告与终点判据
E	原版 Manual 22 页（附录）	保留原 PDF 图、脚本参数和 GUI

> **规则：优先级：**老师最新课堂更新 > 2026 Manual > 本次实际 `CHIP.tcl` / `Route.tcl` 和工具帮助 > 本文教学性补充。不能将一条建议性调试指令误当成老师的必做项。



老师在课堂中公布的 Module 6 更新（真实照片）：Sec.13 免做；Sec.5 的 Routing Layer 改名。

老师更新	原手册	考试时以此为准
第 5 节 SRoute 层界	MET1(1) / MET4(4)	改成 **M1(1) / C2(4)**（三处 Layer Range 均统一）
第 13 节 Parasitic Extraction	rcOut -spef ...	**免做**；不应为提取 SPEF 失败反复消耗考试时间

A | 考试前 5 分钟：硬环境与防呆清单

A1. 先区分两个不同的命令提示符

提示符示例	环境	能执行的命令
[msc_tum_grp2@icd3pc19 Module6]$	Linux shell (csh/bash)	cd, ls, grep, zcat, tail, find, source（注意 shell 类型）
innovus 12>	Innovus 内部 Tcl console	init_design, globalNetConnect, floorPlan, place_design, ccopt_design, timeDesign, saveDesign
design_vision>	Synopsys Design Vision / Tcl	report_power, report_timing 等工具特定命令

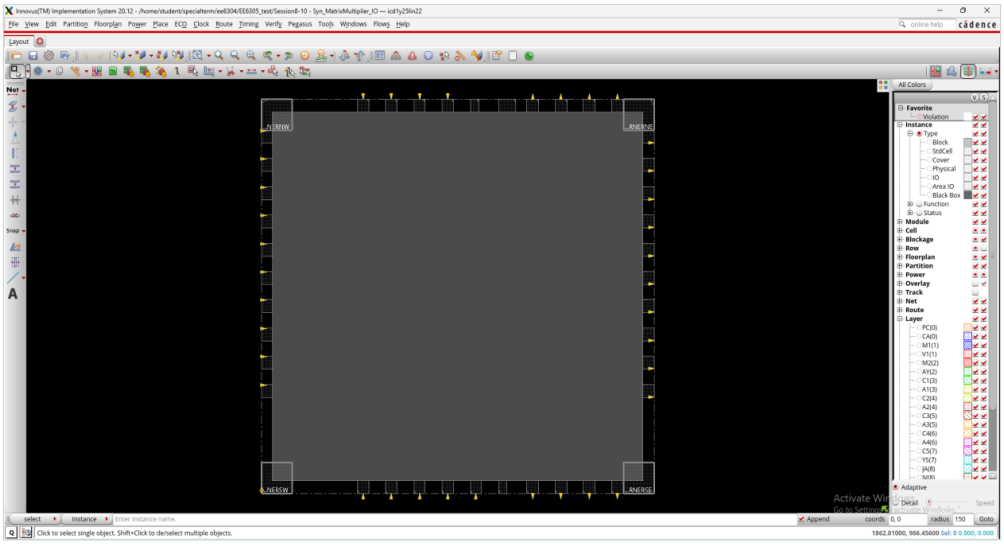
> **高频错误：**Linux shell 看到 `setExtractRCMode: Command not found`、`report_saif: Command not found`，首先怀疑**敲错终端**，不是版图坏了。

A2. 进入 Module6，确认文件和启动方式

Linux terminal；csh 的 source 必须在 csh 中执行

```
cd ~/NM6008/Module6
pwd
ls -l cshrc_inno Syn_MatrixMultiplier_IO.globals Syn_MatrixMultiplier_IO.view
ls -ld IO GDS RC Netlist LIB scripts output_netlist
ls -lh scripts/CHIP.tcl scripts/Route.tcl
# 如果还在 bash/sh，按 Manual 约定切到 csh 再 source：
csh
source cshrc_inno
which innovus
innovus
```

> **检查点：**看到 `innovus 1>` 并进入 Layout GUI，才算进入 EDA console。若 source 的路径不对，先找文件，不要猜安装路径。



原版手册的 Import Design 画面；实际可用命令行 source + init_design，二者是两种入口。

A3. 检查输入文件；绝不要覆盖唯一能恢复的数据库

Linux shell；示例目录操作

```
pwd
ls -lh Netlist/Syn_MatrixMultiplier.v \
       Netlist/Syn_MatrixMultiplier_IO.v \
       Netlist/Syn_MatrixMultiplier_IO.sdc \
       Syn_MatrixMultiplier_IO.globals
find save_design -maxdepth 1 -type f -print
# 新一轮考试先创建自己的备份目录，避免覆盖老师给的文件
mkdir -p checkpoints logs timingReports output_netlist
```

手册 P3 指定了一组示例文件名；如果真实考试题的顶层名不叫 `Syn_MatrixMultiplier_IO`，需要同步改顶层、SDC、I/O、输出前缀，不能机械粘贴。

防误操作：任何 ECO、Filler 或新 Routing 之前：先用 `saveDesign` 保留 checkpoint。`saveDesign` 不等于 Linux shell 下直接敲数据库路径；先识别 `innovus >`。

A4. 只用以下简单标准判断能否进入下一阶段

阶段	允许继续的证据	不允许继续的情况
Import/Floorplan	顶层正确、IO/Power Nets 可见	顶层找不到/缺库/Pin 映射不正确
Placement	place_design 成功、checkPlace 无致命错误	Placement 失败或未保存
CTS	CCOpt 完成、已测 Setup/Hold	Clock Tree 异常/提示 Error
Routing	NanoRoute 完成、无未连接关键信号	Routing 失败、DRC Short 未处理
Final	Setup WNS≥0，Hold WNS≥0，DRC=0，导出文件存在	只有旧报告 PASS、当前数据库仍有 ECO 修改

B｜照 Manual 一步一步完成 Module 6

用法：只执行当前一步的命令，检查“预期结果”，保存检查点，再进入下一步。脚本中的 GUI 录制固定坐标命令在别的设计上可能不适用；本节特别标注。

第 1 步｜Import Design / 初始化（Manual P4–5）

目的：加载 GF22 的库、Verilog、I/O、SDC 和 MMMC/视图数据。

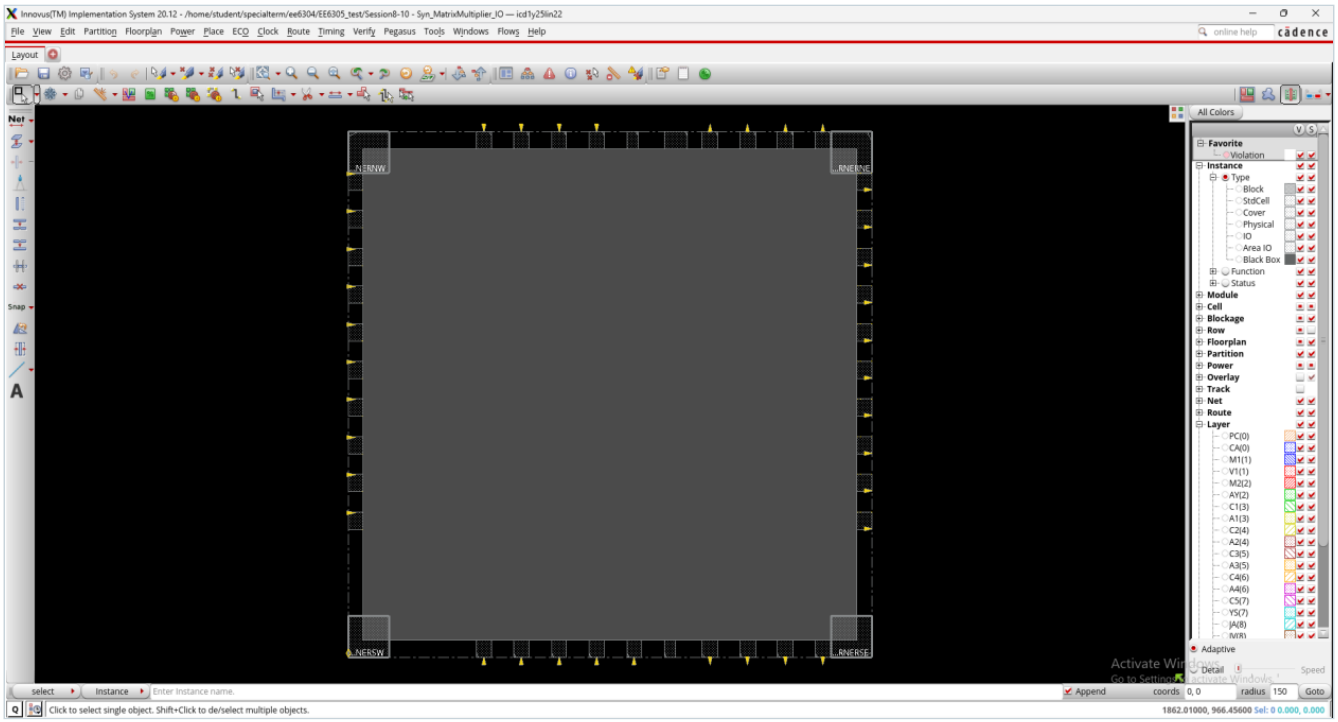
GUI 对应：File → Import Design → Load → Syn_MatrixMultiplier_IO.globals；或用 Tcl。

Innovus Tcl

```
source Syn_MatrixMultiplier_IO.globals
init_design
```

检查点：Layout 中出现 IO Pads / 核心区域，Console 未出现 0 以外的 error；顶层 Syn_MatrixMultiplier_IO。

容易出错：如果报 No related standard cells / no antenna / timing units warning，应查看 ERROR/WARNING 汇总，警告不等于失败。先确认 globals 的路径和库文件是否存在。



Manual P4–5 - Import Design / 初始化 的原版操作示意

第 2 步｜Global Net Connection（Manual P5）

目的：把实例的 VDD/VSS 引脚对应到全局 vdd/gnd 网络（仅定义连线规则，尚非物理电源布线）。

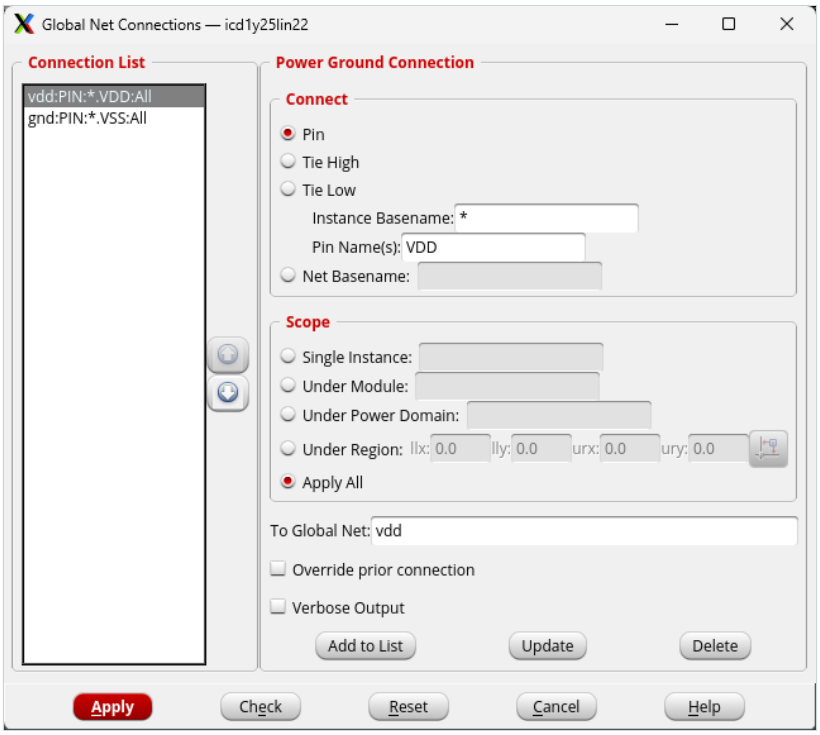
GUI 对应：Power → Connect Global Nets...；选择 PG Pin，指定 Instance=*，Power nets=vdd/gnd。

Innovus Tcl

```
clearGlobalNets
globalNetConnect vdd -type pgpin -pin VDD -inst *
globalNetConnect gnd -type pgpin -pin VSS -inst *
```

检查点：GUI 连接列表中看到 vdd:VDD 和 gnd:VSS；Apply 生效。

容易出错：别把 VDD/VSS 大小写、vdd/gnd 网络名搞反；只有 connect rule 不代表 power route 已完成。



Manual P5 - Global Net Connection 的原版操作示意

第 3 步｜Floorplanning（Manual P6–7）

目的：确定 Die/Core 大小、边界 margin 和合法放置 Row；本例为老师给定 1400x1400。

GUI 对应：Floorplan → Specify Floorplan；按 Manual GUI 填 Size / Margin / Site。

Innovus Tcl

```
getIoFlowFlag
setIoFlowFlag 0
floorPlan -site GF22_SST -d 1400 1400 150 150 150 150 -noSnapToGrid
```

检查点：Die 边缘 IO Pads 形成一圈；中央 Core 正常，`-d` 单位由 Innovus 数据库设定，不应自行认定是 mm。

容易出错：1400×1400 是本实验尺寸，不是通用芯片模板。若 Core 占比很低，可能是参数设定过大；本次 0.327% 逻辑密度并非“Place 没运行”。



Manual P6–7 - Floorplanning 的原版操作示意

第 4 步｜Create Power Ring（Manual P7–9）

目的：在 Core 周边为 vdd/gnd 建立供电 Ring。

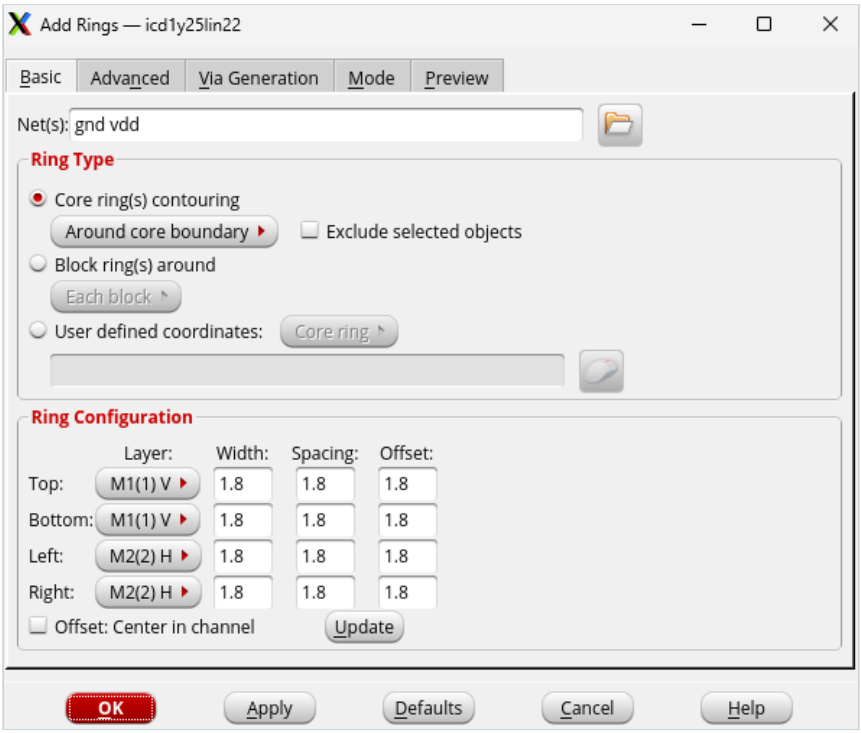
GUI 对应：Power → Power Planning → Add Ring...；Nets={gnd vdd}；Top/Bottom=C2，Left/Right=C1；Width/Spacing/Offset=1.8。

Innovus Tcl

```
setAddRingMode -ring_target default -extend_over_row 0 \
  -ignore_rows 0 -avoid_short 0 -skip_crossing_trunks none \
  -stacked_via_top_layer LB -stacked_via_bottom_layer M1 \
  -via_using_exact_crossover_size 1 -orthogonal_only true \
  -skip_via_on_pin {standardcell} -skip_via_on_wire_shape {noshape}
addRing -nets {gnd vdd} -type core_rings -follow core \
  -layer {top C2 bottom C2 left C1 right C1} \
  -width {top 1.8 bottom 1.8 left 1.8 right 1.8} \
  -spacing {top 1.8 bottom 1.8 left 1.8 right 1.8} \
  -offset {top 1.8 bottom 1.8 left 1.8 right 1.8} \
  -center 0 -threshold 0 -jog_distance 0 \
  -snap_wire_center_to_grid None
```

检查点：Core 周边出现连续 Ring。检查 Ring 的网络/层是否正确，而非只看颜色。

容易出错：首次考试推荐 GUI 填写，或直接读取老师给的 CHIP.tcl 同段脚本；不要自己编造 metal 名称。



Manual P7–9 - Create Power Ring 的原版操作示意

第 5 步｜Special Nets Routing (SRoute)（Manual P9–10）

目的：连接 Ring、Pad Ring、Pad Pins、Core Pins 等电源地路径。

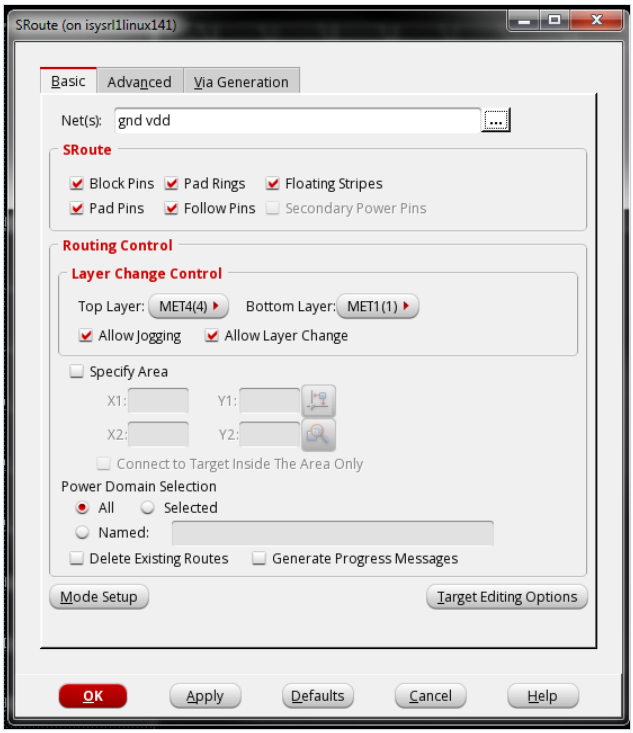
GUI 对应：Route → Special Route...：Nets 填 `gnd vdd`，Block Pins / Pad Pins / Pad Rings / Core Pins / Floating Stripes，Layer Range 下限 M1、上限 C2；可 Jogging / Layer Change。

Innovus Tcl

```
setSrouteMode -viaConnectToShape {noshape}
sroute -connect {blockPin padPin padRing corePin floatingStripe} \
  -layerChangeRange {M1(1) C2(4)} \
  -blockPinTarget {nearestTarget} -padPinPortConnect {allPort oneGeom} \
  -padPinTarget {nearestTarget} -corePinTarget {firstAfterRowEnd} \
  -floatingStripeTarget {blockring padring ring stripe ringpin blockpin followpin} \
  -allowJogging 1 -crossoverViaLayerRange {M1(1) C2(4)} \
  -nets {gnd vdd} -allowLayerChange 1 -blockPin useLef \
  -targetViaLayerRange {M1(1) C2(4)}
```

检查点：从供电 PAD 到 Ring，再到 Core 供电网络可观察到实际连接。

容易出错：老师特别更新：Manual P10 印的是 MET1(1)/MET4(4)，必须统一改成 M1(1)/C2(4)。Manual 后面 selectWire + editResize 的三组坐标来自特定实验布局，不能在坐标不匹配时盲敲。



Manual P9–10 - Special Nets Routing (SRoute) 的原版操作示意

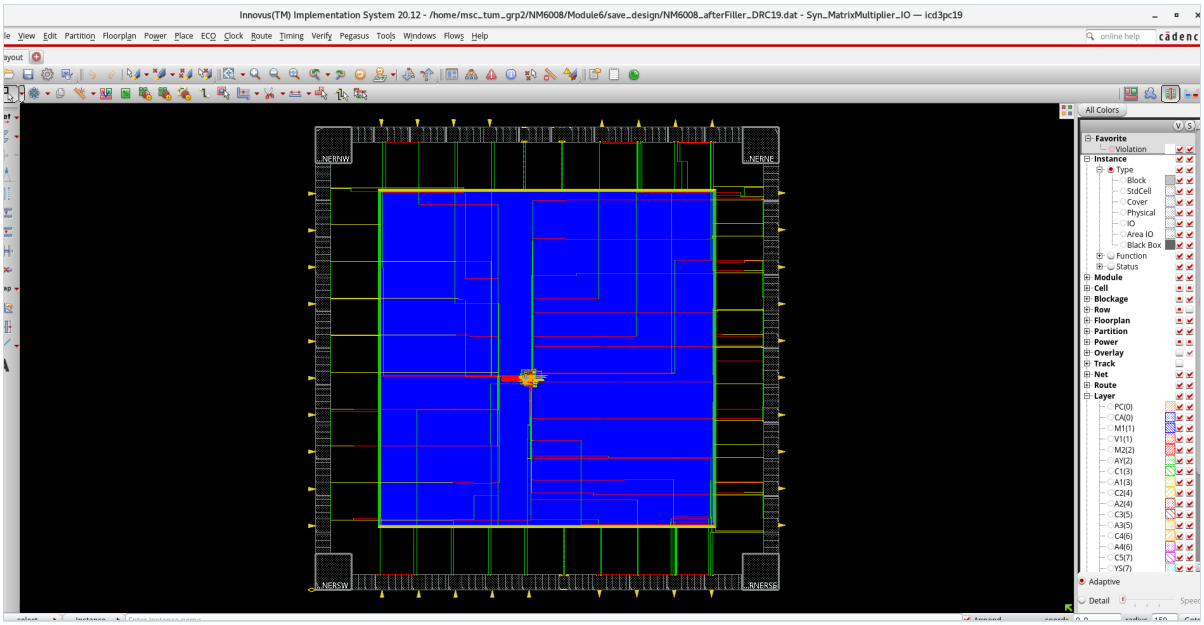
Step 5.1｜老师脚本中的 “拉伸电源线” 如何处理?

Manual P10 给出三组 `selectWire / editResize / editTrim` 命令，目的通常是延长所选的供电金属。**这是坐标操作**，不是电源规则的通用抽象。对于这次课程已提供的特定布局可依手册执行；考试换题，先确认所选 wire 的净名/位置、范围和层。

Innovus Tcl

```
# 仅示例老师给出的第 1 组；执行前请在 GUI 选中并检查目标 net
selectWire 628.9500 1354.0000 633.0500 1354.1180 3 gnd
uiSetTool stretchWire
editResize -direction y -offset -148.259 -side low -keep_center_line auto
deselectAll
```

红线：操作不匹配时：不要无视 `selectWire` 失败而继续 `editResize`；先截图 GUI 和 Console，恢复之前的 checkpoint。



真实实验记录：Core Ring、SRoute 与部分互连的可视化（历史中间 checkpoint）。

第 6 步｜I/O Pad Filler Placement（Manual P11）

目的：把 IO Pad 间隙填充，形成连续的 pad frame，满足本实验物理规则。

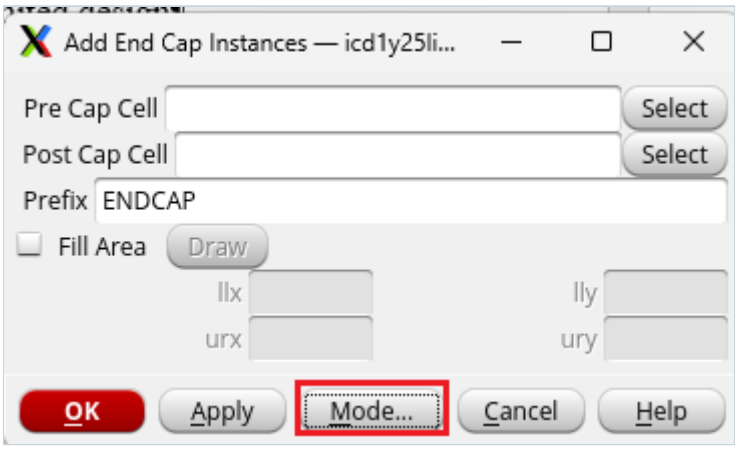
GUI 对应：Place → Physical Cell / Add I/O Filler（如有菜单差异，以本机工具界面为准）；从大到小依序选择。

Innovus Tcl

```
addIoFiller -cell "IN22FDX_GPIO18_10M19S40PI_FILL20_V IN22FDX_GPIO18_10M19S40PI_FILL10_V
IN22FDX_GPIO18_10M19S40PI_FILL5_V IN22FDX_GPIO18_10M19S40PI_FILL1_V" -prefix IOFILLER_N -side n
addIoFiller -cell "IN22FDX_GPIO18_10M19S40PI_FILL20_V IN22FDX_GPIO18_10M19S40PI_FILL10_V
IN22FDX_GPIO18_10M19S40PI_FILL5_V IN22FDX_GPIO18_10M19S40PI_FILL1_V" -prefix IOFILLER_S -side s
addIoFiller -cell "IN22FDX_GPIO18_10M19S40PI_FILL20_H IN22FDX_GPIO18_10M19S40PI_FILL10_H
IN22FDX_GPIO18_10M19S40PI_FILL5_H IN22FDX_GPIO18_10M19S40PI_FILL1_H" -prefix IOFILLER_W -side w
addIoFiller -cell "IN22FDX_GPIO18_10M19S40PI_FILL20_H IN22FDX_GPIO18_10M19S40PI_FILL10_H
IN22FDX_GPIO18_10M19S40PI_FILL5_H IN22FDX_GPIO18_10M19S40PI_FILL1_H" -prefix IOFILLER_E -side e
```

检查点：四个边的 pad 间隙由不同宽度 IO Fillers 填满；应确认实际单元存在于 IO LEF/Lib。

容易出错：不要使用 Core Filler 命令代替 IO Filler；N/S 的 `_V` 与 E/W 的 `_H` 与老师示例一致。



Manual P11 - I/O Pad Filler Placement 的原版操作示意

第 7 步｜End Cap Placement（Manual P11–12）

目的：修复 StdCell Row 两端的 Well/边界物理要求。

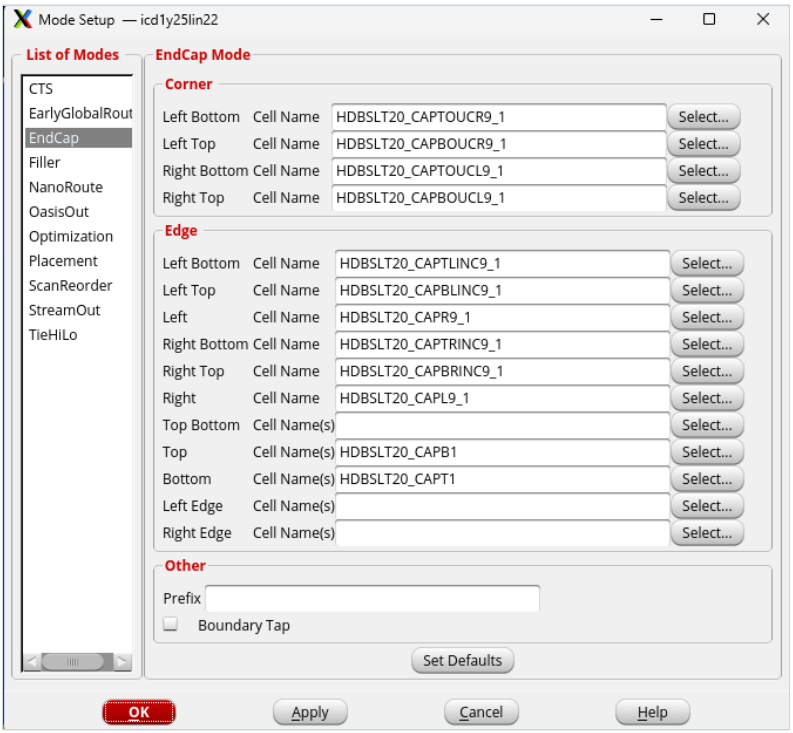
GUI 对应：Place → Physical Cell → Add End Cap... → Mode；按老师给出的 GF22 型号选。

Innovus Tcl

```
snapFPlan -all
set CELL_PREFIX "HDBSLT20"
setEndCapMode -reset
setEndCapMode \
  -topEdge "${CELL_PREFIX}_CAPB1" \
  -bottomEdge "${CELL_PREFIX}_CAPT1" \
  -leftEdge "${CELL_PREFIX}_CAPR9_1" \
  -rightEdge "${CELL_PREFIX}_CAPL9_1" \
  -rightTopCorner "${CELL_PREFIX}_CAPBOUCL9_1" \
  -rightBottomCorner "${CELL_PREFIX}_CAPTOUCL9_1" \
  -leftTopCorner "${CELL_PREFIX}_CAPBOUCR9_1" \
  -leftBottomCorner "${CELL_PREFIX}_CAPTOUCR9_1" \
  -leftBottomEdge "${CELL_PREFIX}_CAPTLINC9_1" \
  -lefttopEdge "${CELL_PREFIX}_CAPBLINC9_1" \
  -rightBottomEdge "${CELL_PREFIX}_CAPTRINC9_1" \
  -righttopEdge "${CELL_PREFIX}_CAPBRINC9_1" \
  -fitGap true
addEndCap -prefix ENDCAP
```

检查点：可见行首/行尾 End Cap，Add End Cap 返回成功，电源连接与 row 方向无明显冲突。

容易出错：原 Manual P12 截图的引号中有若干尾随空格；不能凭空证明它们被工具忽略。这里显示经清理的可读命令，考试优先对照实验室同版 `CHIP.tcl` 的 cell 名与库实际存在情况。



Manual P11–12 - End Cap Placement 的原版操作示意

第 8 步｜Standard Cell Placement（Manual P13）

目的：将实例放进 Core Row，并进行初步拥塞/线长评估。

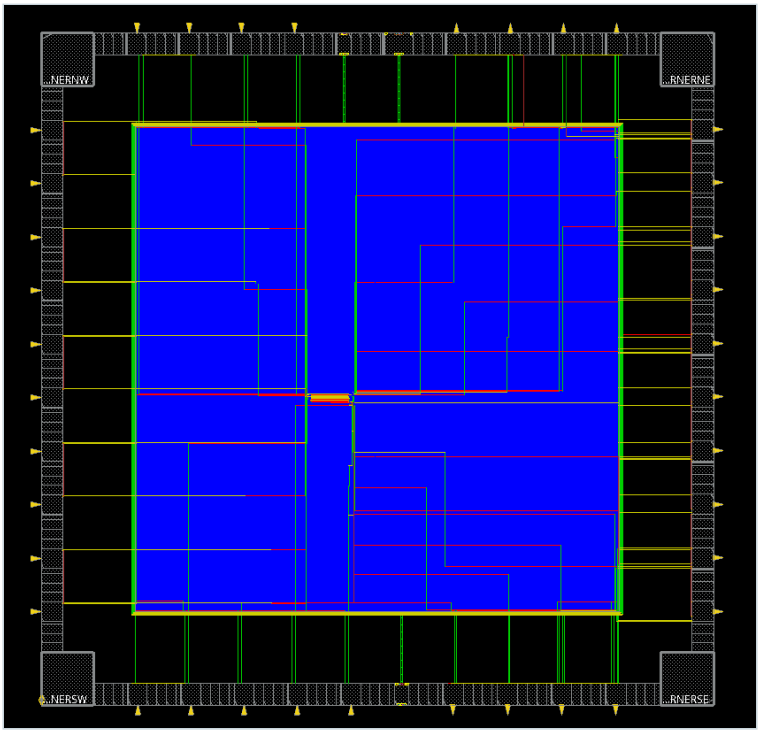
GUI 对应：Place → Place Standard Cell... → Apply。

Innovus Tcl

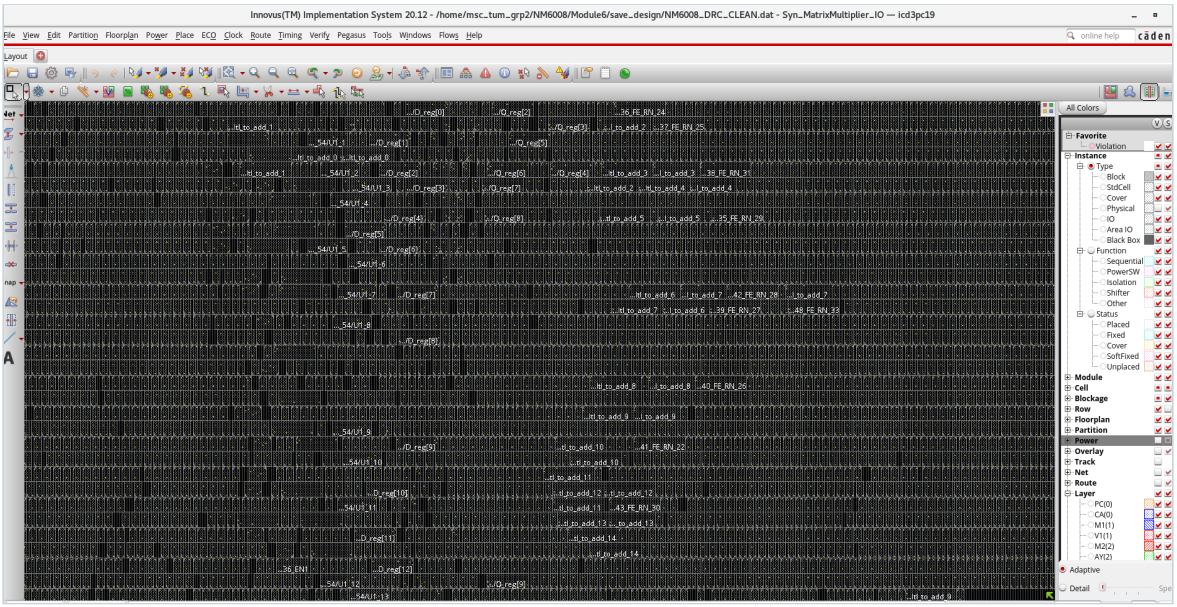
```
setPlaceMode -fp false
place_design
# 你的 CHIP.tcl 中还出现：
checkPlace CHIP_TOP.checkPlace
saveDesign save_design/MatrixMultiplier_post_place
```

检查点：Placement 完成、没有未放置功能单元，`checkPlace` 有输出，保存 Post-Placement Checkpoint。

容易出错：Manual P13 印的是 `placeDesign`；你实际 `CHIP.tcl` 用的是 `place_design`。选择本机已识别的命令，不能不核对就认定二者总是等价。



Manual P13 - Standard Cell Placement 的原版操作示意



我们实际展示的标准单元/填充行近景：为了看清实例已隐藏布线，非独立 Placement 阶段截图。

第 9 步｜Pre-CTS Timing / Opt / CTS（Manual P14–17）

目的：先在理想时钟前提下检查 Setup；再建真实时钟树并检查 Skew、Slew、Insertion Delay。

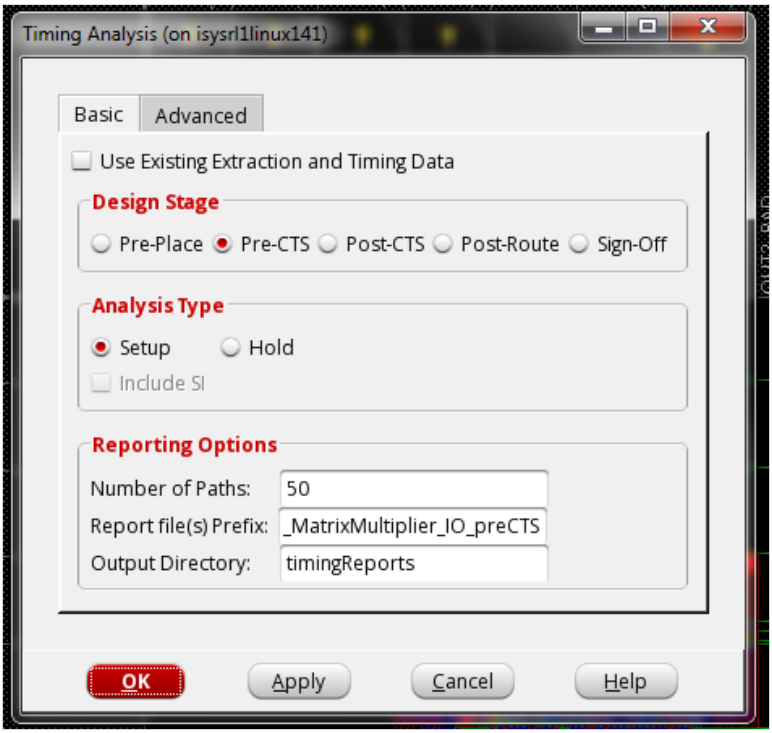
GUI 对应：Timing → Report Timing → Pre-CTS / Setup；ECO → Optimize Design → Pre-CTS；Clock → CCOpt Clock Tree Debugger。

Innovus Tcl

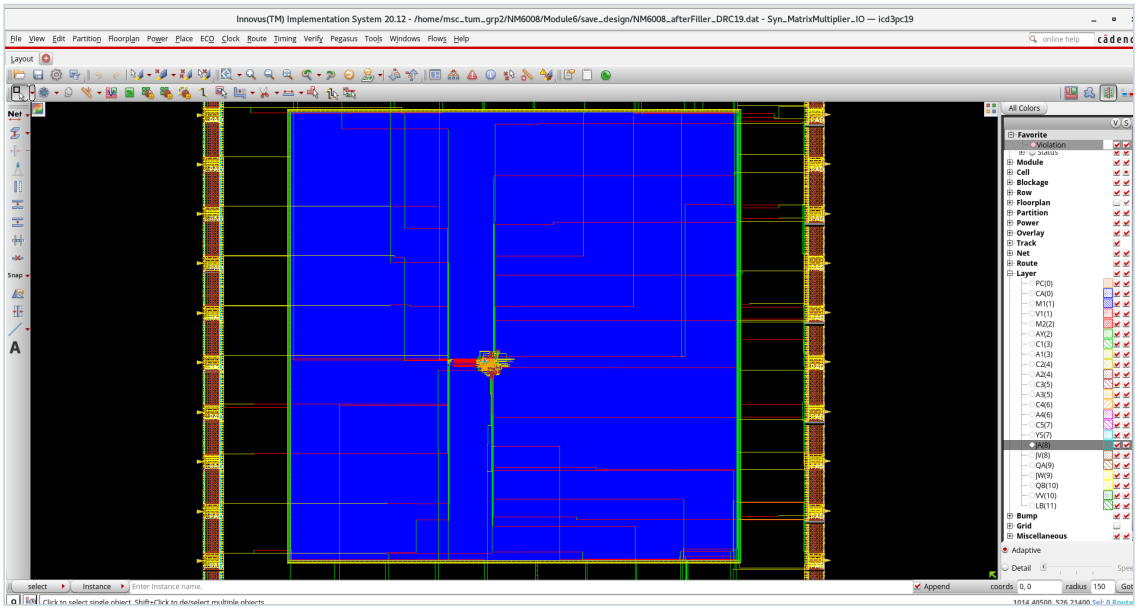
```
timeDesign -preCTS -idealClock -pathReports -drvReports \
  -slackReports -numPaths 50 -prefix Syn_MatrixMultiplier_IO_preCTS \
  -outDir timingReports
setOptMode -fixCap true -fixTran true -fixFanoutLoad false
optDesign -preCTS
create_ccopt_clock_tree_spec
ccopt_design -prefix cts
ctd_win -id ctd_window
```

检查点：Pre-CTS Setup 报告显示 WNS/TNS/Violating Paths；CTS 后 GUI Clock Tree Debugger 可看到 root→buffer→sink。

容易出错：Pre-CTS 不能修 Hold；Hold optimization 发生在 Post-CTS。不要只看 Clock Tree GUI，就认为 Hold 已经通过。



Manual P14–17 - Pre-CTS Timing / Opt / CTS 的原版操作示意



实际 Clock Tree Debugger 截图：根节点、buffer 分支、寄存器 sinks。

Step 9.1｜Post-CTS Setup + Hold 优化（本实验最关键！）

Manual P17 明确要求 post-CTS Setup/Hold 检查、必要时优化；你在 `scripts/CHIP.tcl` 第 120–138 行执行的标准流程如下。先 check，再 opt，再复测：

Innovus Tcl

```
timeDesign -postCTS -pathReports -drvReports -slackReports \
  -numPaths 50 -prefix postCTS_setup_before -outDir timingReports
setOptMode -fixCap true -fixTran true -fixFanoutLoad true
optDesign -postCTS

timeDesign -postCTS -hold -pathReports -slackReports \
  -numPaths 50 -prefix postCTS_hold_before -outDir timingReports
setOptMode -fixCap true -fixTran true -fixFanoutLoad true
optDesign -postCTS -hold
optDesign -postCTS -drv

timeDesign -postCTS -pathReports -drvReports -slackReports \
  -numPaths 50 -prefix postCTS_setup_after -outDir timingReports
timeDesign -postCTS -hold -pathReports -slackReports \
  -numPaths 50 -prefix postCTS_hold_after -outDir timingReports
```

检查点：先别以单次 Hold 优化结束：DRV 再优化可能改变 Hold；必须使用新前缀输出 Setup 与 Hold 复测。你之前出现 53→5→21 的变化就是例子。

实验阶段	Hold WNS	违规条数
Post-CTS 初次	−2.492 ns	53
Hold 优化后	约 −0.001 ns	5
DRV 优化后	−0.014 ns	21
最终 Post-Route	+0.419 ns	0

第 10 步｜NanoRouting / Detail Routing（Manual P17–19）

目的：Global Routing 做布线资源规划，Detail/NanoRoute 最终落实 Track、Metal、Via 和完整连线。

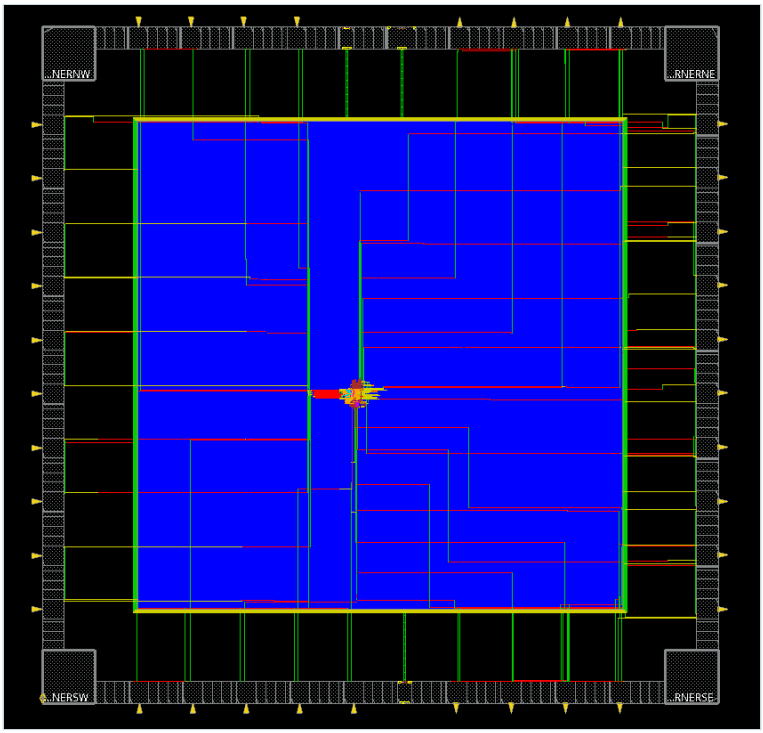
GUI 对应：Route → NanoRoute（按实验环境菜单）；本实验主要从 Tcl script source。

Innovus Tcl

```
# 仅在到达 routing 阶段、且确认当前数据库与脚本预期一致时：
source ./scripts/Route.tcl
```

检查点：Routing 结束并产生可观察的信号连线；随后需要 Post-Route Setup/Hold/DRV 分析。

容易出错：`Route.tcl` 可能包含初始化、修复或重新 route 操作，不能在已有完整 routing + filler 的数据库中无条件运行第二遍。先阅读文件内容。



Manual P17–19 - NanoRouting / Detail Routing 的原版操作示意

Step 10.1｜按 Manual 做 Post-Route Timing

Innovus Tcl

```
# 若需要改变分析模式，优先按手册 P18 的 GUI 设置 OCV；
# 使用工具支持的 Tcl 参数时必须在 innovus >，而不是 Linux shell
setExtractRCMode -engine postRoute -effortLevel signoff \
  -coupled true -capFilterMode relAndCoup -coupling_c_th 0.1 \
  -total_c_th 0 -relative_c_th 1 \
  -lefTechFileMap ./GDS/22FDSOI_10M_2Mx_5Cx_1Jx_2Qx_LB_qrc_leftech.map
# Manual P18 的 setAnalysisMode -analysisType onChipVariation 也可由 GUI 设置
# 如果该命令的精简用法在本机不支持，请以完整 Manual 参数 / help 为准。
setAnalysisMode -cppr none -analysisType onChipVariation

timeDesign -signoff -pathReports -drvReports -slackReports \
  -numPaths 50 -prefix final_route -outDir timingReports
timeDesign -signoff -hold -pathReports -slackReports \
  -numPaths 50 -prefix final_route -outDir timingReports
```

限制：本实验曾尝试 Signoff 级 Quantus RC Extraction，但 CentOS 7 的 GLIBC 不兼容。**不要假定 `timeDesign -signoff` 的报告等同于 Quantus 完成的独立寄生参数签核。**老师已通知 Sec.13 免做。

第 11 步｜Core Filler Placement / DRC 修复（Manual P19–20）

目的：填充标准单元间隙，为 Well/电源连续和局部 DRC 提供物理结构。

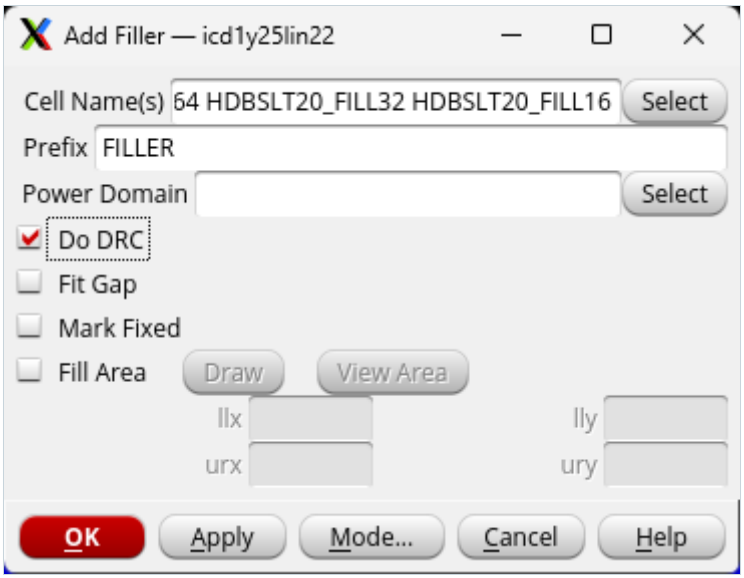
GUI 对应：Place → Physical Cell → Add Filler... → 勾 Do DRC。

Innovus Tcl

```
getFillerMode -quiet
addFiller -cell HDBSLT20_FILL64 HDBSLT20_FILL32 \
  HDBSLT20_FILL16 HDBSLT20_FILL8 HDBSLT20_FILL4 \
  HDBSLT20_FILL3 HDBSLT20_FILL2 HDBSLT20_FILL1 \
  -prefix FILLER -doDRC
# 老师原脚本中在 filler 后还执行一次 Route.tcl；
# 本文不建议在未检查脚本内容时自动重复 source。
```

检查点：Core Filler 填充完成，应运行 DRC 检查，关注 Short/Spacing 等真实违规。

容易出错：我们出现过 19 条 DRC（其中 7 Short），repair 后剩 1 条 M1 Short；最终使用针对性的 routing ECO 清除。不可只看 addFiller 返回成功就判定最终 DRC=0。



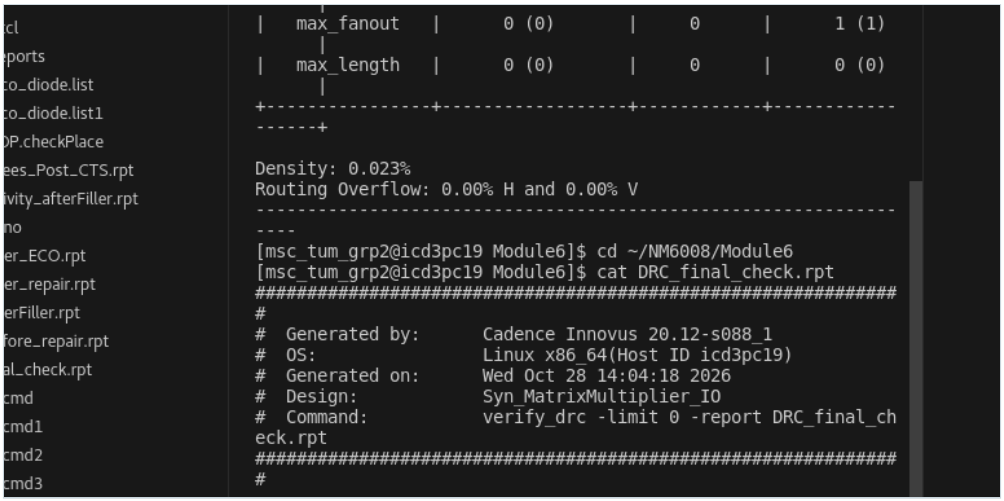
Manual P19–20 - Core Filler Placement / DRC 修复 的原版操作示意

Step 11.1｜本实验真正解决 DRC 的方式

Innovus Tcl

```
# 当前数据库先保存，绝不能以覆盖唯一旧 checkpoint 的方式修复
saveDesign save_design/before_DRC_repair
verify_drc -limit 0 -report DRC_before_repair.rpt
# 本机课程流程通过 addFiller 的修复/DRC 选项收敛绝大部分问题
# 残余 M1 metal short 再针对性 ECO：
ecoRoute -fix_drc
verify_drc -limit 0 -report DRC_after_ECO.rpt
# 确认零违规后才保存：
saveDesign save_design/NM6008_DRC_CLEAN
```

检查点：后验事实：19→1→0 是本实验的实际结果，**不是**说明 `ecoRoute -fix_drc` 对所有 short 都可以“一键包治”。如果修复后仍有 Short，应根据 DRC 报告精确定位 Geometry，再决定人工修复/恢复 checkpoint。



真实最终 DRC 文本证据：“No DRC violations were found”。

第 12 步｜GDSII Stream-Out（Manual P20–21）

目的：导出可被版图交换工具识别的几何图层（非功能仿真文件）。

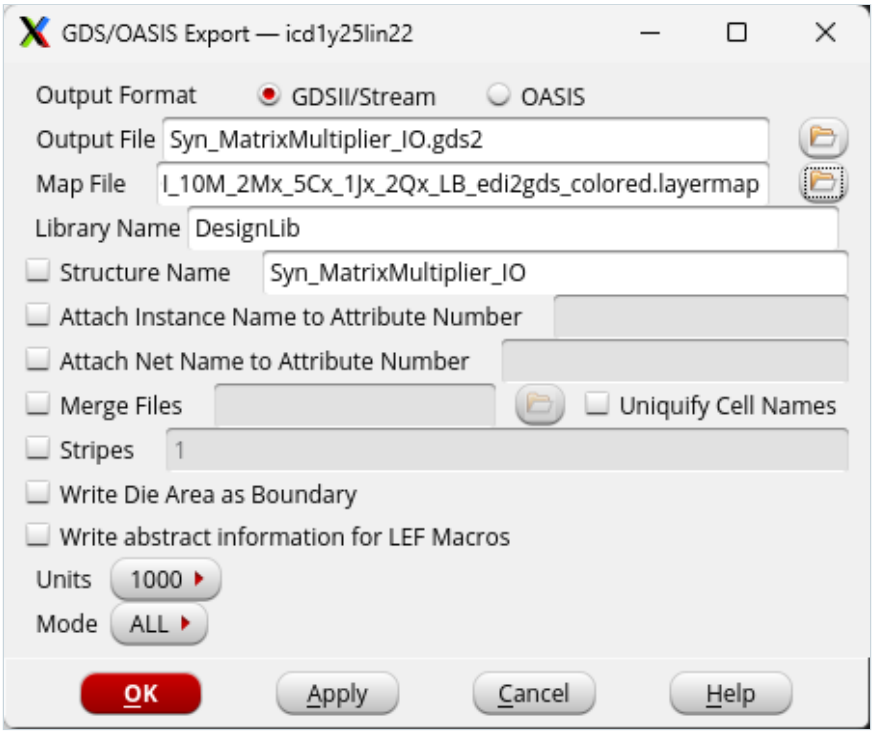
GUI 对应：File → Save → GDS/OASIS…；选 GDSII、输出文件、Layer Map、Structure Name。

Innovus Tcl

```
streamOut Syn_MatrixMultiplier_IO.gds2 \
  -mapFile GDS/22FDSOI_10M_2Mx_5Cx_1Jx_2Qx_LB_edi2gds_colored.layermap \
  -libName DesignLib -structureName Syn_MatrixMultiplier_IO \
  -units 1000 -mode ALL
```

检查点：Console 出现 Streamout finished；Linux 下 `ls -lh Syn_MatrixMultiplier_IO.gds2` 非零，报告约 22 MB。

容易出错：GDSII 的 layer map 与 SRoute 的 M1/C2 层范围是不同概念；不要把一个映射文件用于另一个命令。



Manual P20–21 - GDSII Stream-Out 的原版操作示意

第 13 步｜Parasitic Extraction（2026 老师免做）（Manual P21）

目的：原手册讲解将金属/过孔寄生 R/C 导出 SPEF；但本组课程更新明确免做。

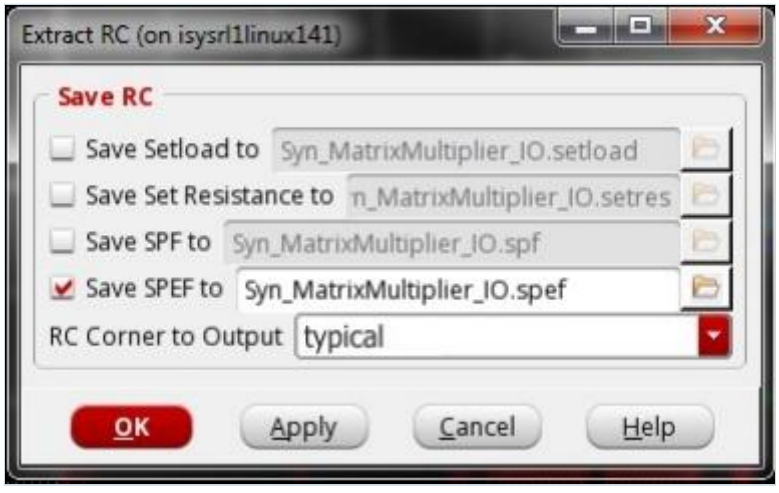
GUI 对应：原 Manual：Timing → Extract RC...。老师更新：本环节跳过，不需要创建 SPEF。

Innovus Tcl

```
# 仅供识别，2026 更新后不要作为必做：
# rcOut -spef Syn_MatrixMultiplier_IO.spef -rc_corner typical
```

检查点：在最后报告中写明 “Section 13 waived per instructor update”；不要虚报 SPEF 已成功导出。

容易出错：我们的实际报错：IMPEXT-7050、NREX-94、GLIBC_2.25/2.27/2.28 not found。确认是外部 Quantus 24.10 与 CentOS 7 GLIBC 不匹配；不是 `qrcTechFile` 的文本第 205 行错误。



Manual P21 - Parasitic Extraction（2026 老师免做） 的原版操作示意

第 14 步｜SDF Timing File Generation（Manual P21）

目的：输出 Standard Delay Format，供后布局/门级仿真做 SDF timing annotation。

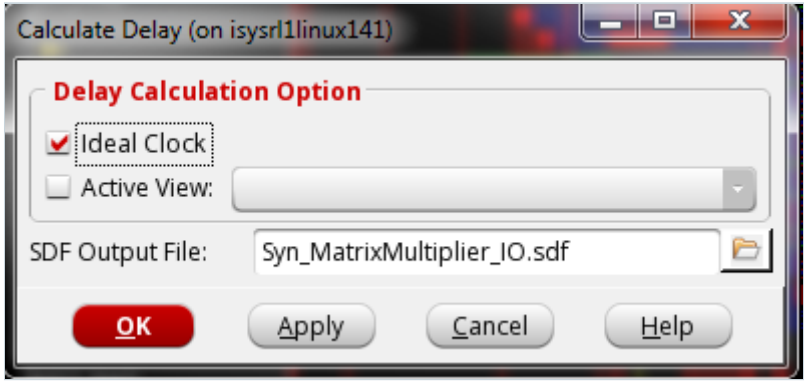
GUI 对应：Timing → Write SDF...；选择输出文件及正确 timing view。

Innovus Tcl

```
all_hold_analysis_views
all_setup_analysis_views
write_sdf Syn_MatrixMultiplier_IO.sdf
```

检查点：`ls -lh` 显示非空 SDF；本实验最终约 3.7 MB。

容易出错：SDF 是延迟标注，不是 SPEF RC 网络；没有 SPEF 并不自动意味着 SDF 未产生，必须单独检查 SDF 命令及其内容。



Manual P21 - SDF Timing File Generation 的原版操作示意

第 15 步｜Post-Layout Verilog Netlist（Manual P22）

目的：输出布局修改后的标准单元门级网表，以供后续 Gate-Level Simulation。

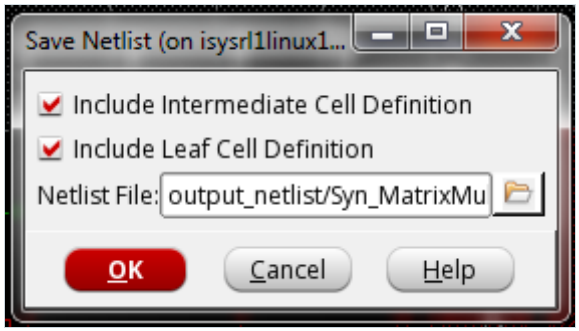
GUI 对应：File → Save → Netlist...；确认输出路径。

Innovus Tcl

```
saveNetlist output_netlist/Syn_MatrixMultiplier_IO.v
```

检查点：输出文件非空；顶层 `module Syn_MatrixMultiplier_IO` 存在；本实验约 1.3 MB。

容易出错：不要把 P&R 网表当成最初综合 RTL；Post-Route Gate-Level Simulation 时需匹配相应 libraries + SDF + testbench。



Manual P22 - Post-Layout Verilog Netlist 的原版操作示意

C｜我们真正遇到的 18 类故障：症状 → 原因 → 处理

按**可搜索的原始报错关键词**编排。先核对你实际所在 shell 和当前 checkpoint。对没有被真实日志证明彻底解决的环境问题，本手册只给出诊断及停机条件，不虚构成功案例。

D01｜innovus: Command not found

现象：在普通 Linux shell 敲 `innovus`，程序未找到。

根因/判断：没有在 csh 中 source cshrc_inno，或 PATH / 许可证未加载。

正确处理：切 `csh` → `source cshrc_inno` → `which innovus`；不要把 Innovus 工具指令敲进普通 shell。

> **复现时的诊断/修复命令（注意终端）**
>
> ```
> csh
> source cshrc_inno
> which innovus
> ```

D02｜setExtractRCMode: Command not found

现象：在 `[... Module6]$` 后敲 setExtractRCMode / optDesign 等。

根因/判断：工具命令被放在 Linux shell，而非 `innovus N>`。

正确处理：返回 Innovus Tcl console，检查提示符。若 Innovus 因 Error/Killed 退出，先重新 source 环境并 restoreDesign。

> **复现时的诊断/修复命令（注意终端）**
>
> ```
> pwd
> ps -ef | grep innovus
> ```

D03｜report_saif: Command not found

现象：在 Linux shell 敲 `report_saif -hierarchy`。

根因/判断：属于 Synopsys Design Compiler / Power Compiler 工具域，既不是普通 shell 命令，也不是 Module6 Innovus 的必做。

正确处理：打开正确的 Synopsys shell；Module6 后端不需要 report_saif 才能完成。

复现时的诊断/修复命令（注意终端）

```
# 不要在 Linux shell 运行 report_saif
```

D04｜No related standard cells / No antenna...

现象：启动 init_design 时出现大量 WARN，最终显示 2890 warnings / 0 errors。

根因/判断：库中存在非致命技术单元/Antennas 相关警告及 timing units 警告；不是所有 WARNING 都致命。

正确处理：优先筛查 ERROR、缺失关键库、最终初始化状态；如果 GUI/设计顶层正确再继续。

复现时的诊断/修复命令（注意终端）

```
grep -nEi "ERROR|WARNING|IMPTS-16" innovus.log* | tail -40
```

D05｜setAnalysisMode: Command not found

现象：在 Linux terminal 用 `setAnalysisMode -analysisType onChipVariation`。

根因/判断：同 D02，终端错误；也可能 GUI/工具版本对精简参数支持不同。

正确处理：进入 Innovus Console；`help setAnalysisMode` 或查看手册 P18 的完整参数，并核对菜单 Tools → Set Mode。

复现时的诊断/修复命令（注意终端）

```
# Innovus Tcl
help setAnalysisMode
```

D06｜file exists *.map -> 0

现象：执行 `file exists 22FDSOI...leftech.map` 返回 0。

根因/判断：工作路径在 Module6，但真实文件在 `./GDS/`。

正确处理：用 `find` 定位路径，再在 `setExtractRCMode -lefTechFileMap` 中使用相对/绝对真实路径。

复现时的诊断/修复命令（注意终端）

```
find . -iname "*leftech*" -print
# Innovus Tcl
file exists ./GDS/22FDSOI_10M_2Mx_5Cx_1Jx_2Qx_LB_qrc_leftech.map
```

D07｜NREX-94: 11 routing layers, 1 mapped

现象：extractRC 失败，提示设计 11 层但只有 1 层匹配到 techfile。

根因/判断：工艺映射/RC extraction 环境或配置不一致；仅看到 map 文件并不足以证明映射正确。

正确处理：先核对 RC corner、QRC techfile 和 layer map 是否对应，不能删除完整 layermap 后继续强行跑；本课程 Section 13 已免做。

复现时的诊断/修复命令（注意终端）

```
# Innovus Tcl
getExtractRCMode
```

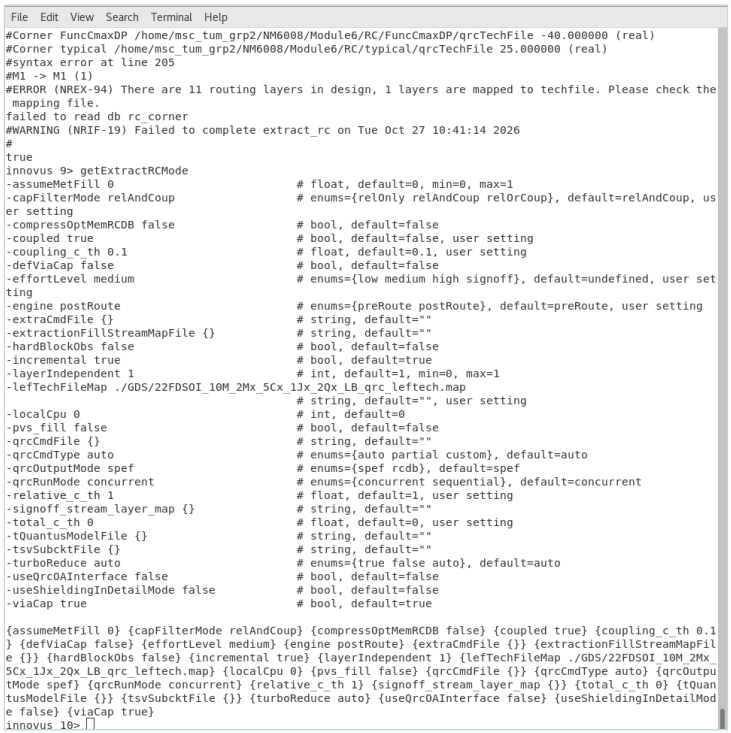
D08 | GLIBC_2.27 / GLIBC_2.28 not found

现象：Quantus `qrc` 报 libc.so.6 和 libm.so.6 缺少 GLIBC_2.25/2.27/2.28。

根因/判断：CentOS 7 使用 GLIBC 2.17；运行的 Quantus 24.10 需要更高符号版本。

正确处理：不要擅自替换系统 libc；在本实验按老师更新跳过 Step 13；若生产项目必须提取，联系管理员提供兼容工具/受支持 OS。

> **复现时的诊断/修复命令（注意终端）**
>
> ```
> ldd --version | head -n 1
> ```



历史截图：Quantus/RC 错误的真实 Terminal / GUI 场景；关键错误文字以本页可搜索文字为准。

D09 | qrcTechFile contains garbage at line 205

现象：用 `nl -ba RC/typical/qrcTechFile` 显示乱码。

根因/判断：qrcTechFile 是二进制格式（文件实测约 141 MB），不是普通 Tcl/ASCII 文本。

正确处理：不用 `sed/nl` 修改/解释其内容；文件存在也不代表提取引擎兼容。

> **复现时的诊断/修复命令（注意终端）**
>
> ```
> file -L RC/typical/qrcTechFile
> ls -lh RC/typical/qrcTechFile
> ```

D10 | IMPEXT-7050: in-memory changes

现象：`rcOut -spef` 报 `Writing parasitics can not be done as there are in-memory changes...`。

根因/判断：RC 数据处于修改中，必须完成一致的提取才可导出；但当前 Quantus 无法启动。

正确处理：保留输出，记录为环境阻断；不要把前次的旧 SPEF 误当当前 ECO 数据；本实验 Sec.13 不必继续。

复现时的诊断/修复命令（注意终端）

```
# 2026 本实验已免做 rcOut
```

D11｜LS_PLACEOPT::runCurrentState internal failure

现象：`optDesign -postRoute -hold` 报 `PR:doExtrInit`。

根因/判断：调用外部提取阶段失败，与后面的 QUANTUS/GLIBC 兼容性错误相关；不能仅据此判断 Netlist 错。

正确处理：检查对应时间戳附近的 ERROR、GLIBC/Quantus 日志；使用稳定 PreRoute/PostCTS 优化阶段并按手册做复测，避免无意义重复。

复现时的诊断/修复命令（注意终端）

```
grep -nEi "LS_PLACEOPT|doExtrInit|GLIBC|QUANTUS|ERROR" innovus.logv* | tail -50
```

D12｜Post-CTS Hold WNS -2.492, 53 paths

现象：PreCTS Setup 正，但 PostCTS Hold 大量违规。

根因/判断：原 SDC `set_clock_uncertainty 2.5` 给 Hold 带来很苛刻要求，部分寄存器路径非常短、delay cells 被 dont_use。

正确处理：用 `timeDesign -postCTS -hold` 精确定位，`optDesign -postCTS -hold` 修复，再测 Setup、DRV；最后在 Post-Route 以新报告验收。不能直接改老师 SDC 来“消除违规”。

复现时的诊断/修复命令（注意终端）

```
timeDesign -postCTS -hold -pathReports -slackReports -numPaths 50 -prefix hold_before -outDir
timingReports
optDesign -postCTS -hold
```

D13｜Hold 5 -> 21 paths after DRV

现象：Hold opt 后 WNS 约 -0.001；DRV 后 WNS -0.014、21 Paths。

根因/判断：DRC/DRV/优化过程会改变 cell/route/timing；一次修复不是最终结果。

正确处理：同时输出新 Setup/Hold Summary；继续按老师脚本完成路由并以最终报告判断。

复现时的诊断/修复命令（注意终端）

```
timeDesign -postCTS -hold -pathReports -slackReports -numPaths 50 -prefix afterDRV -outDir timingReports
```

D14｜DRC 19 violations, 7 Shorts

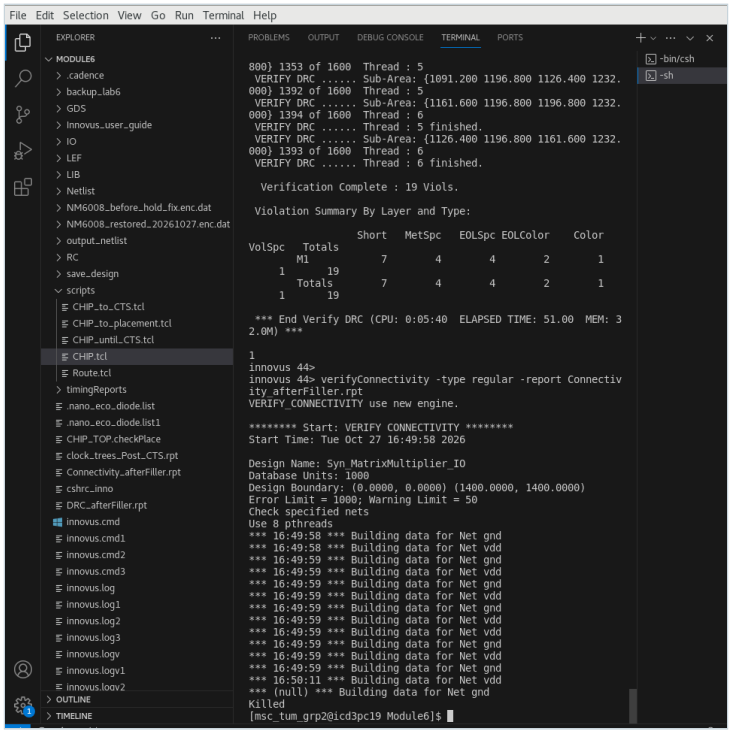
现象：verify_drc 检出 M1 Short、MetSpc、EOLSpc 等。

根因/判断：Filler/Routing 变化叠加、局部 M1 实际接触/间距问题。

正确处理：先 saveDesign，再 `verify_drc -report ...`；处理 filler 问题后再次检查，不要声称有 GDS 就代表 DRC Pass。

复现时的诊断/修复命令（注意终端）

```
saveDesign save_design/before_repair
verify_drc -limit 0 -report DRC_before_repair.rpt
```



历史截图：Filler 后 DRC 检查和多种违规的环境画面。

D15｜1 M1 Short near (641.014,622.350)

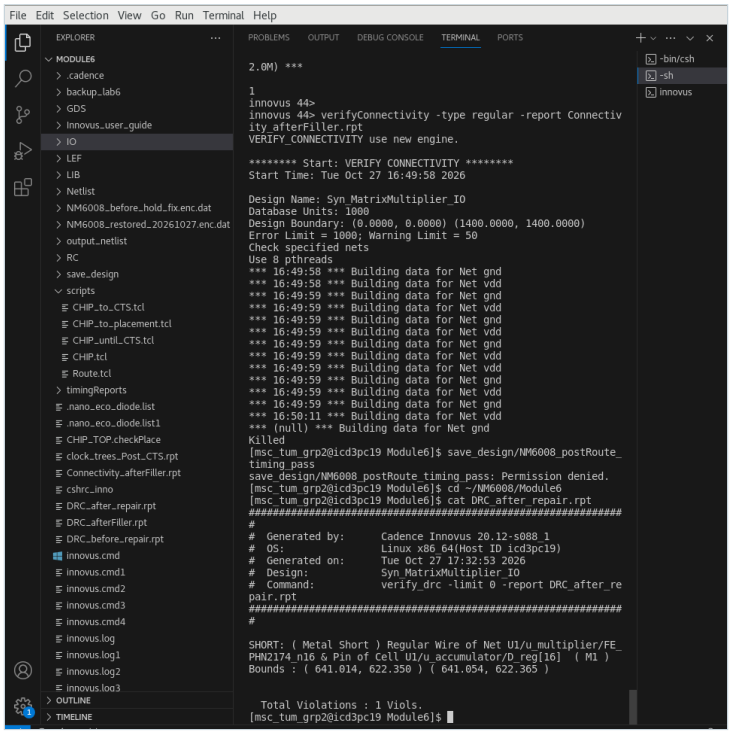
现象：Short between `U1/u_multiplier/FE_PHN2174_n16` wire and `U1/u_accumulator/D_reg[16]` Pin。

根因/判断：这是信号线与标准单元 Pin 局部碰撞，非简单“多加 Filler”即可修复。

正确处理：保留恢复点，按局部 ECO route 修复，然后重新 `verify_drc`，实际本次收敛为 0。

复现时的诊断/修复命令（注意终端）

```
saveDesign save_design/NM6008_DRC1_before_ECO
ecoRoute -fix_drc
verify_drc -limit 0 -report DRC_after_ECO.rpt
```



历史截图：一次修复后只剩一个 M1 Short 的操作阶段。

D16｜verifyConnectivity ... Killed

现象：验证供电网时不断 `Building data for Net gnd/vdd` 后进程被系统 Killed。

根因/判断：常见为内存资源耗尽或系统调度中断；截图本身不能独立证明唯一根因。

正确处理：检查 `free -h` / 任务负载 / dmesg（若有权限）；重启会话后恢复 checkpoint；不要把 Killed 写成 Connectivity PASS。

> **复现时的诊断/修复命令（注意终端）**
>
> ```
> free -h
> # 如果有权限：dmesg | tail -30
> ```

D17｜save_design/... Permission denied

现象：直接在 Linux shell 输入 `save_design/NM6008_postRoute_timing_pass`。

根因/判断：shell 把相对路径当可执行文件。

正确处理：改到 `innovus N>` 使用 `saveDesign save_design/...`；若用 Linux 列文件则 `ls -l`。

> **复现时的诊断/修复命令（注意终端）**
>
> ```
> # Innovus Tcl
> saveDesign save_design/NM6008_safe_checkpoint
> ```

D18｜saveDesign / restoreDesign Command not found

现象：在 Linux shell 或混淆 Tcl session 执行 saveDesign/restoreDesign。

根因/判断：工具命令必须在 Innovus shell，且恢复设计名的 `*.dat` 与数据库头文件配套。

正确处理：从 Linux 正确启动 Innovus；在其 console 用 `restoreDesign`，必须确认恢复的是 DRC_CLEAN 而不是 DRC19 旧版本。

复现时的诊断/修复命令（注意终端）

```
# Innovus Tcl
restoreDesign save_design/NM6008_DRC_CLEAN.dat Syn_MatrixMultiplier_IO
```

D｜最后 10 分钟验收 / 恢复 / 签核

D1. 先确定数据库是最终 DRC-Clean 版本

Innovus Tcl

```
# Linux shell 只用来查看目录
cd ~/NM6008/Module6
ls -lh save_design/NM6008_DRC_CLEAN*
# Innovus Tcl Console 中执行（不能在 Linux shell 下执行）：
restoreDesign save_design/NM6008_DRC_CLEAN.dat Syn_MatrixMultiplier_IO
# 根据工具当前菜单恢复 GUI 版图视图，f = Fit Design（在版图区）
```

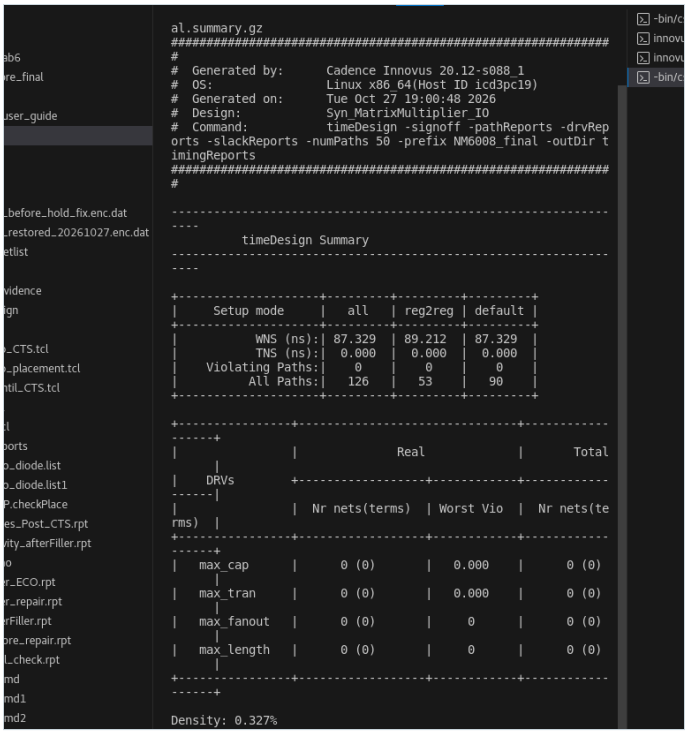
检查点：不要在标题栏出现 `NM6008_afterFiller_DRC19.dat` 的画面截“Final DRC Clean Layout”；同理，单独一张空白 Core 图不能证明成功布线。

D2. 保存最终 Setup / Hold / DRC 检查证据

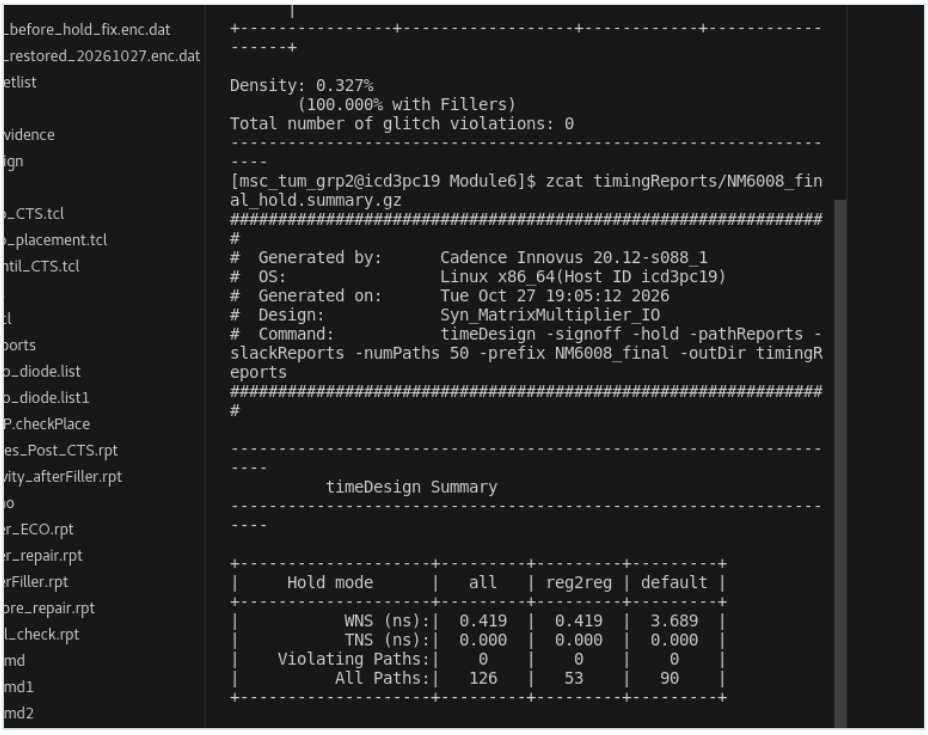
Linux shell 读报告；这些是本次实际文件名

```
# Innovus Tcl
# 以下 timing 报告已在本实验生成；考试只有在当下数据库一致时才可重跑
# 若以最新报告为准，Linux shell 用 zcat 阅读即可
# Linux shell：
cd ~/NM6008/Module6
zcat timingReports/NM6008_final.summary.gz
zcat timingReports/NM6008_final_hold.summary.gz
cat DRC_final_check.rpt
```

本次已确认的结果	数值	合格解释
Final Setup WNS	+87.329 ns	TNS 0.000；Violating Paths 0
Final Hold WNS	+0.419 ns	TNS 0.000；Violating Paths 0
Electrical DRV (real)	0	max_cap / max_tran / max_fanout / max_length = 0
Final Innovus DRC	0	报告逐字写明 No DRC violations were found
Density（不含 Filler）	0.327%	不要把 with Filler 的 100% 当逻辑利用率



本实验最终 Setup Summary（原始证据裁图）。



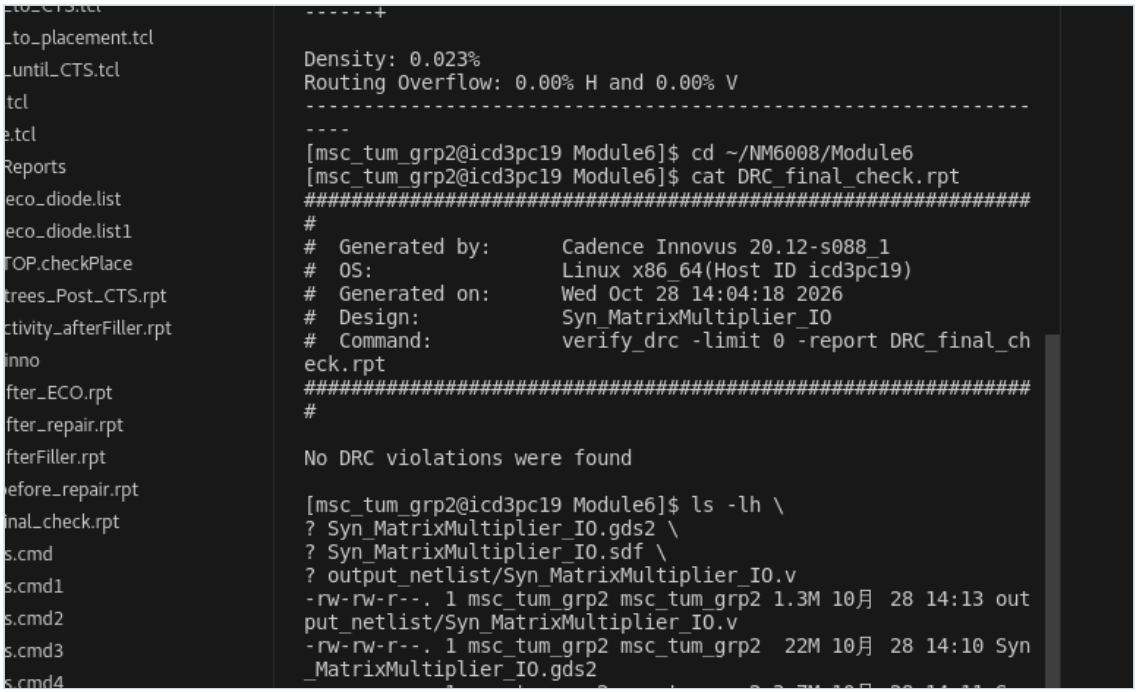
本实验最终 Hold Summary（原始证据裁图）。

D3. 三个交付文件必须存在且非空

Linux shell

```
cd ~/NM6008/Module6
ls -lh Syn_MatrixMultiplier_IO.gds2 \
       Syn_MatrixMultiplier_IO.sdf \
       output_netlist/Syn_MatrixMultiplier_IO.v
grep -nE '^[[:space:]]*module[[:space:]]+Syn_MatrixMultiplier_IO([[:space:]]|\()' \
   output_netlist/Syn_MatrixMultiplier_IO.v | head
```

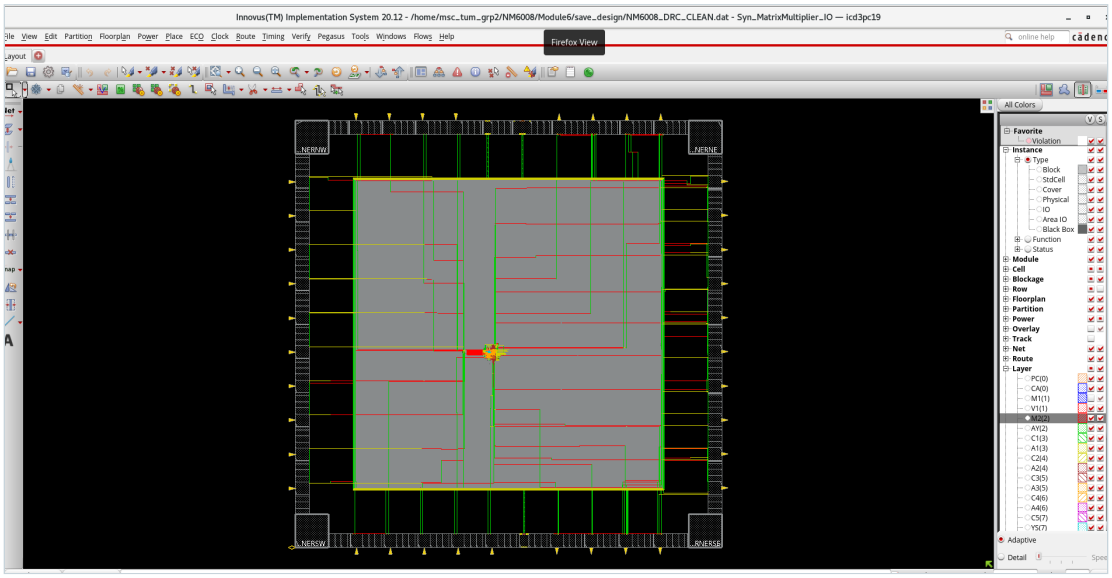
检查点：已记录的文件约为：GDSII 22 MB；SDF 3.7 MB；Netlist 1.3 MB。文件大小并非通用 PASS 标准，且必须对应最终 ECO 后的设计。



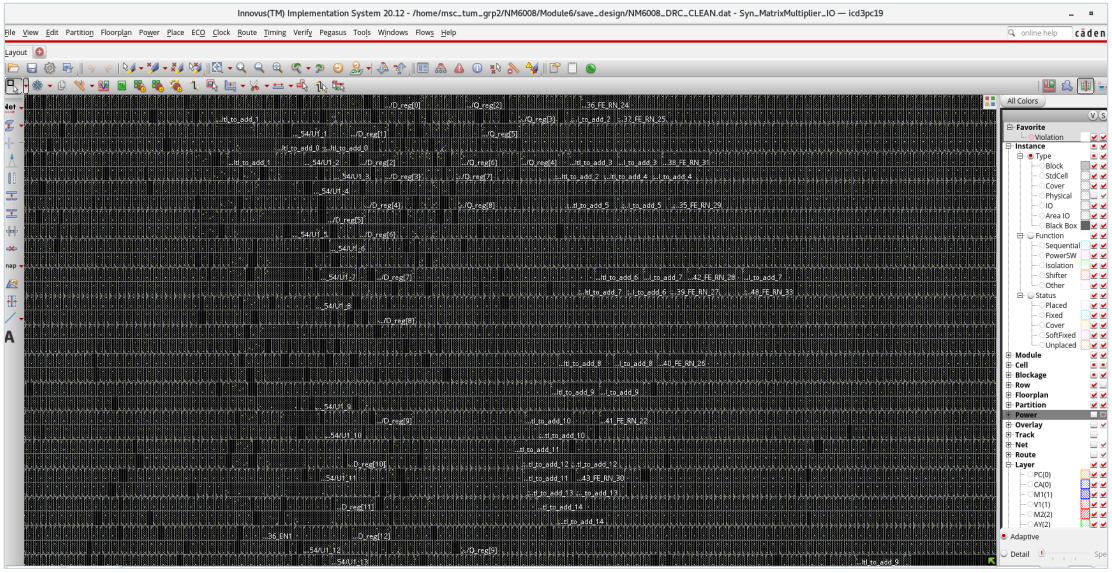
真实最终导出文件证据：GDSII、SDF 与 Verilog Netlist。

D4. 标准截图表（考试交报告按此收集）

编号	需截取的画面	有效的证据形式
01	Import / Floorplan	可见 IO Pads + Core；文件/顶层正确
02	Power Ring / SRoute	Core Ring + Pad/Core 供电连接
03	Placement	隐藏遮挡金属，看得见 StdCell 行和 label
04	Clock Tree Debugger	root + buffers + sinks
05	Pre-CTS Setup Report	带 WNS/TNS/Violating Paths
06	Final Routing	明确是最后保存的 DRC_CLEAN checkpoint
07	Final Setup	Post-Route / Signoff 命令及 Summary
08	Final Hold	对应最新 ECO 数据库的 Hold Summary
09	Final DRC	`No DRC violations were found` 可读
10	Final files	GDSII/SDF/Netlist 三项完整文件名和大小



最终恢复 DRC_CLEAN 后，隐藏 M1 遮挡得到的可视化 Routing/Layout（展示视图不是完整所有 Metal）。



隐藏 Routing 后查看 StdCell / Filler placement 行的真实界面。

D5. GUI 可见性开关（考试最容易误把“看不见”当“没有”）

Innovus 右侧 **All Colors** 分为 V（Visible）与 S（Selectable），必须只操作 V 列来控制截图。Placement 视图隐藏 Net/Route/Power/部分 Layer；Routing 视图显示 Net/Route/Power 和适当金属层。**不要修改真正数据库以改善截图效果。**

D6. 10 条绝不能犯的错误

- 在 Linux shell 里敲 Innovus Tcl / Design Vision 的命令。
- 忘记 `csh` + `source cshrc_inno`，直接宣称 Innovus 不可用。
- 把 Manual P10 旧 MET1/MET4 层名继续用于 2026 更新版。
- 把老师原本 Section 13 的 SPEF 步骤当作考试必须完成。
- 未保存 checkpoint 就执行再次 Placement、CTS 或 Route。

- 在输出中只看到 WARNING 就认定整个工具失败，或看到 0 ERROR 就认定 Timing/DRC 全 PASS。
- 只看 Setup WNS>0，忽略 Hold、Short、DRV 和连通性。
- 用旧 checkpoint 的 DRC/Timing 结果签核最新 ECO 数据库。
- 为消除 Hold 私自改变老师的 SDC 并不说明影响。
- 把 `100% with Fillers` 当成逻辑面积利用率 100%。

D7. 故障恢复三步法

```tcl
# (1) Linux shell 验证数据库存在:
ls -lh save_design/NM6008_DRC_CLEAN*
# (2) 正确启动工具:
csh
source cshrc_inno
innovus
# (3) Innovus Tcl Console 中恢复:
restoreDesign save_design/NM6008_DRC_CLEAN.dat Syn_MatrixMultiplier_IO
```

实操原则：只有确认当前 checkpoint/报告阶段一致，才继续 Routing、DRC、SDF/GDSII/Netlist 导出。恢复成功≠必须重跑全部 15 步。

E｜原始 Manual 作为 PDF 附录的使用说明

后附 2026 版《NM6008 Module 6 - Layout of Matrix Multiplier Circuit》**原始 22 页**，不删减原来的 GUI 截图、命令及细节。PDF 阅读器内可用 `Ctrl+F` 搜索本节标题和报错。

附录版本说明：警告：附录保持历史原貌，P10 仍印有 MET1/MET4，P21 仍包含 SPEF 提取。**这两处必须以老师照片更新为准**。对脚本具体版本若与手册冲突，先查看你当场获得的文件。

NM6008 Module 6 • SOP End • Original 22-page instructor Manual follows.

NANYANG TECHNOLOGICAL UNIVERSITY
School of Electrical and Electronic Engineering

NTU-TUM MSc (IC Design) Programme
Digital IC Design Course: NM6008

Laboratory Manual

Module 6

Layout of Matrix Multiplier Circuit

Course Instructors: A/Prof Lim Meng Hiot
A/Prof Gwee Bah Hwee
Dr Cheng Deruo
Sheng Shirui
Yu Guanhua
Lee Yujin
Liu Shukun

12nd Oct – 29st Oct 2026

Section 1: Introduction

This tutorial style laboratory manual document is meant for NTU-TUM Master of Science (MSc) course – Integrated Circuit (IC) Design, NM6008. This module introduces the concepts that serve as a bridge between modelling and synthesis to the actual physical layout level of an IC system. The following sub-sections state the objectives, the design tools/process used and the experimental design information of this module – Layout of Matrix Multiplier circuit.

1.1 Objective

Module 6 gives a quick introduction to the Place & Route layout implementation. The design example used is the Matrix Multiplier Circuit. The primary objective of this module is to let students familiarize themselves with the Cadence IC Place & Route layout tool (and the associated design methodology). The specific objectives include:

(i) To complete a layout using the standard library cells; and
(ii) To learn the design methodology of the semi-custom IC layout implementation.

1.2 Integrated Circuits (IC) Design Tools and Process Technology

Students will be exposed to the following IC design tools:

- Cadence Innovus (Cadence)
- Design Vision (Synopsys)

To learn more about these IC design tools, you may wish to read/refer to the following documents:

__, Cadence Innovus 20.1 User Guide, Cadence
__, Design Compiler™ User Guide. Synopsys.

The process technology used is the GF FDSOI 22nm CMOS fabrication process (different from previous technology process we used)

Section 2: Procedures

2.1 Working Environment

1. **Go** to your home directory and **choose** Module6. You will find the following folders:
   - **IO**

- **GDS**
- **RC**
- **Netlist**
- **Scripts**
- **LIB**
- **output_netlist**

Two other files will be available namely:

- **Syn_MatrixMultiplier_IO.globals**
- **Syn_MatrixMultiplier_IO.view**

In the **Netlist and IO** folder, files required to place and route are given. These files are usually provided by foundries or third party vendors depending on the process and standard cell libraries that you are using.

The **netlist** folder contains files below:

- **Syn_MatrixMultiplier.v**
- **Syn_MatrixMultiplier_IO.sdc**
- **Syn_MatrixMultiplier_IO.v**
- **Syn_MatrixMultiplier.io**

The file **Syn_MatrixMultiplier.v** is a Verilog file which is generated upon executing the synthesis of the Matrix Multiplier Circuit developed in Module 5. Alternatively, you can use your Matrix Multiplier Circuit if the input and output of your design are the same as those of the **Syn_MatrixMultiplier.v** here.

The file **Syn_MatrixMultiplier_IO.v** is a Verilog file in which the file **Syn_MatrixMultiplier.v** is instantiated. Input and output pads, input clock pad as well as power pads are added to the instantiated module.

[Note: The file **Syn_MatrixMultiplier_IO.v** is provided for ease of implementation for this lab exercise.]

The power pads are used to provide the power supply to the internal circuitry as well as to connect the internal circuitry to ground. The power pads are often treated as both input and output pads.

The file **Syn_MatrixMultiplier.io** provides IO pad locations for the design.

The **scripts** folder contains a TCL file named script.tcl which contains the sequential commands necessary to develop the complete layout of the matrix multiplier circuit. In the TCL file, the commands written without '#' signs before them indicate the actual valid commands. However the commands written with '#'

symbols are comments which are not executed. These comments serve as useful reminders or provide informative descriptions about the scripts.

The **timing_library** folder contains library timing files used for timing analysis, timing debugging, clock tree synthesis and timing aware routing.

2.2 Using Innovus

1. To open **Innovus**, type the following commands in a terminal:

   >> cd /home/"your home directory name"/Module6
   >> source cshrc_inno
   >> innovus

2. All the valid commands present in the TCL file should be executed in the terminal window sequentially and correspondingly the development of the circuit layout can be observed in the **Innovus** tool. The steps can be observed in Section 3.

Section 3: Steps in Implementing the Layout

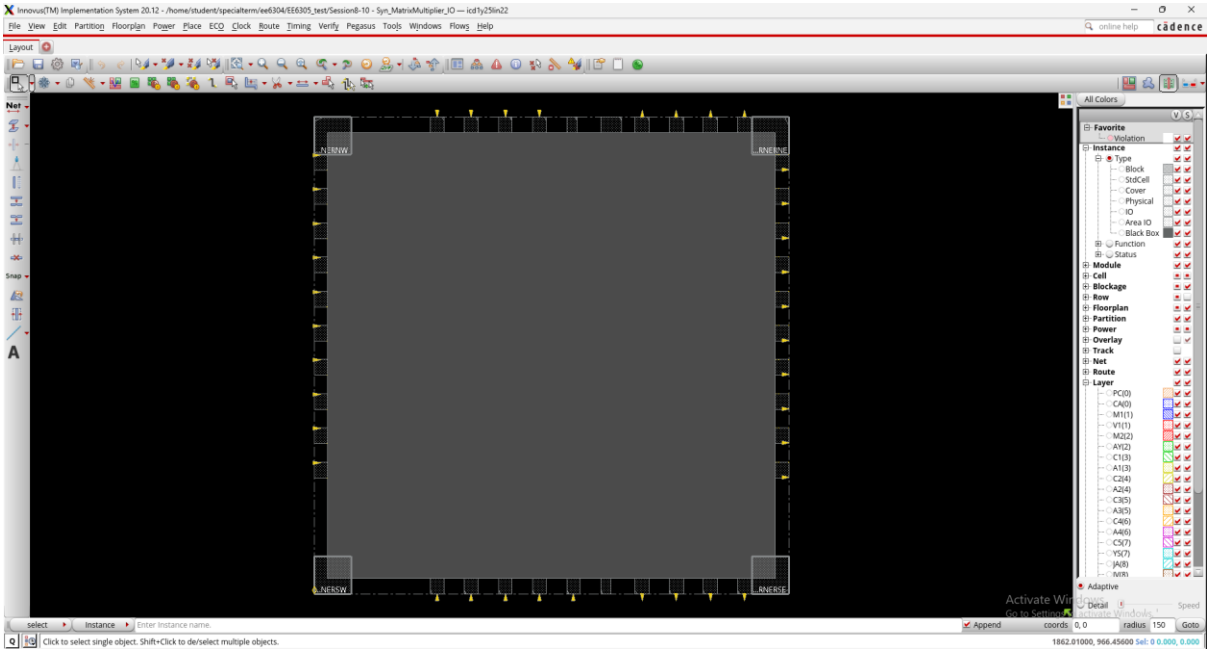
Note: All the commands need to be executed in the Innovus terminal.

1. Loading the global variables file

The first set of commands in the TCL file when executed will load the completed matrix multiplier Verilog code, including the port declarations onto the Innovus tool. The first set of commands is indicated below

```
>> source Syn_MatrixMultiplier_IO.globals
>> init_design
```

Upon execution, you will be able to visualize the core of the circuit present in the middle surrounded by the declared input and output ports as shown below.



Alternatively, you can load global variables file from GUI by clicking "**File**" and then choose "**Import Design…**". Click "**Load…**" in the "Design Import" window and choose `Syn_MatrixMultiplier_IO.globals` file.

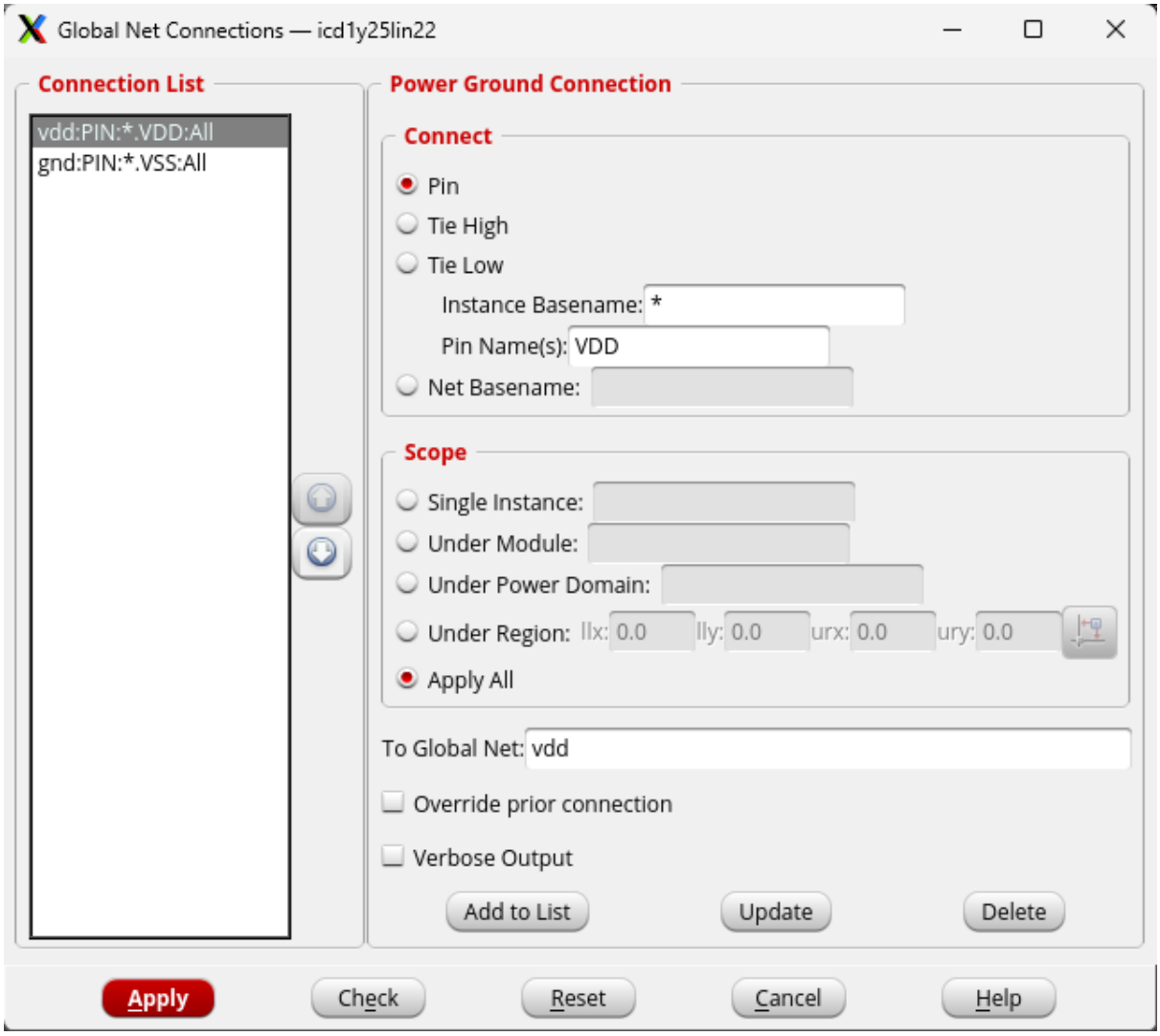
[Remarks: The `Syn_MatrixMultiplier_IO.globals` file links library information and design information. The *.globals* file provides a good reference if you need to setup other *.globals* file for other project.]

2. Global Net Connection

Global net connections specify which power/ground pin (of cells or pads) connects to which global net. The following commands are used to define the global power nets required in order to make global connections:

```
>> clearGlobalNets
>> globalNetConnect vdd -type pgpin -pin VDD -inst *
>> globalNetConnect gnd -type pgpin -pin VSS -inst *
```

To specify global nets from GUI, click on "**Power**" then choose "**Connect Global Nets**…" to open "Global Net Connection" window to add global power nets information. Click "**Apply**" after all information has been specified.



3. Floorplanning

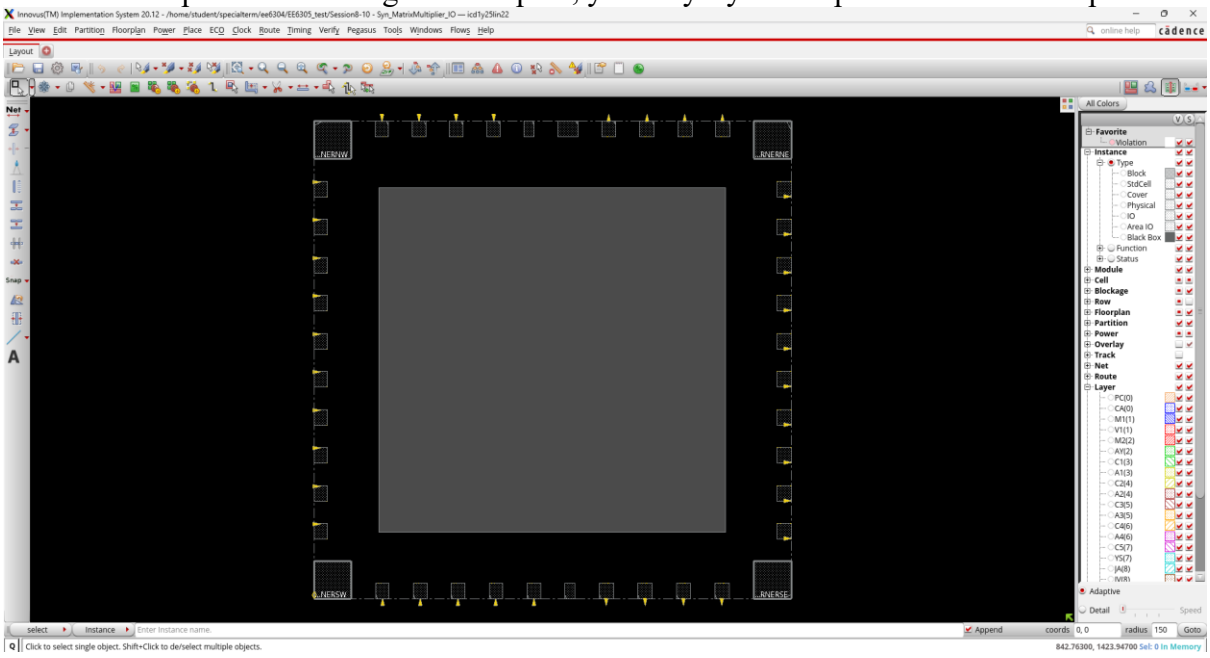
We now need to generate the floorplan for the matrix multiplier circuit. It would be desirable to generate the floorplan by using GUI. From GUI, first click on "**Floorplan**" and then choose "**Specify Floorplan**" to open "Specify Floorplan" window. Provide floorplan information as shown in the picture and click "Apply".



The floorplan can also be modified by using commands below:

```
>> getIoFlowFlag
>> setIoFlowFlag 0
>> floorPlan -site GF22_SST -d 1400 1400 150 150 150 150 -noSnapToGrid
```

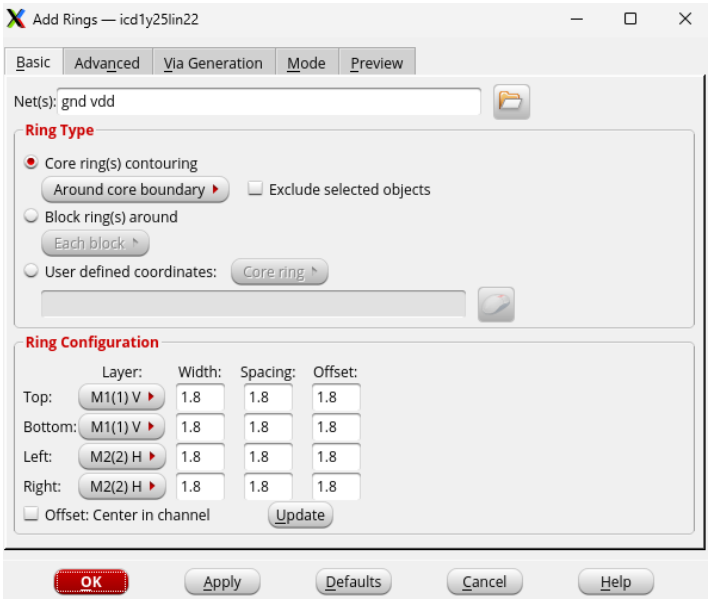
The floorplan is modified based on specification you have provided. Although above-mentioned steps will create a good floorplan, you may try and explore different floorplans.



4. Power Rings Creation

The next step involved in developing the layout is to create a connection between the input/output pads and the internal core circuitry so that the external power supply can be made

available to the internal circuit. TO add power rings from GUI, click on "**Power**", then click "**Power Planning**" and choose "**Add Ring…**" to open up "Add Rings" window. Add the power nets that you would like to connect and specifications for rings before clicking "**OK**" or "**Apply**".

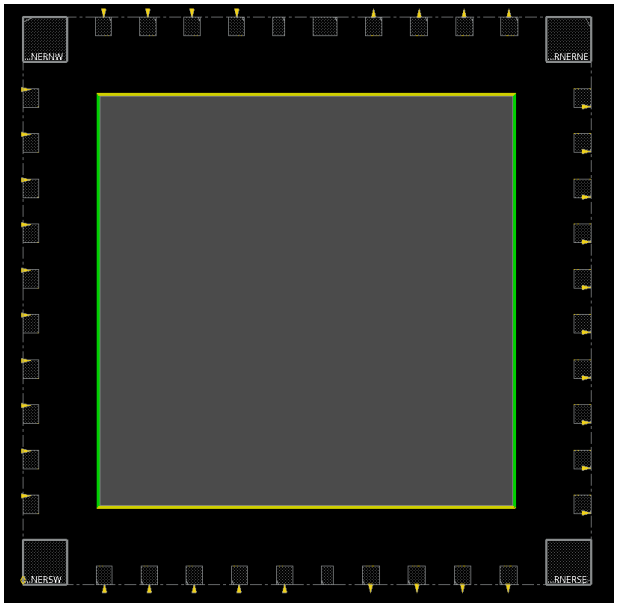


Power rings can also be added by execution of the following commands.

```
>> setAddRingMode -ring_target default -extend_over_row 0 -ignore_rows 0 -avoid_short 0 -skip_crossing_trunks none -stacked_via_top_layer LB -stacked_via_bottom_layer M1 -via_using_exact_crossover_size 1 -orthogonal_only true -skip_via_on_pin { standardcell } -skip_via_on_wire_shape { noshape }

>> addRing -nets {gnd vdd} -type core_rings -follow core -layer {top C2 bottom C2 left C1 right C1} -width {top 1.8 bottom 1.8 left 1.8 right 1.8} -spacing {top 1.8 bottom 1.8 left 1.8 right 1.8} -offset {top 1.8 bottom 1.8 left 1.8 right 1.8} -center 0 -threshold 0 -jog_distance 0 -snap_wire_center_to_grid None
```

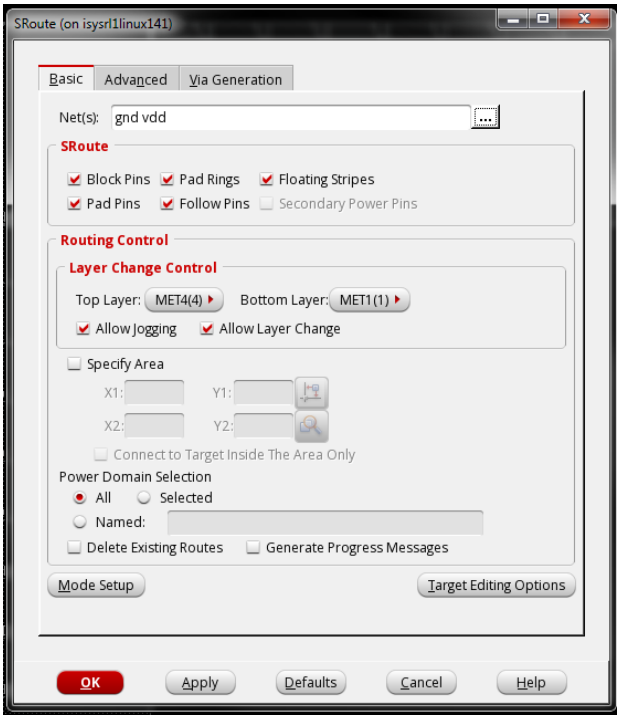
The Innovus window displays the result as shown below:



5. Special Nets Routing

Special nets routing (SRoute) creates power/ground stripes, connects core/block rings to the power/ground pads, and power/ground nets for pads.

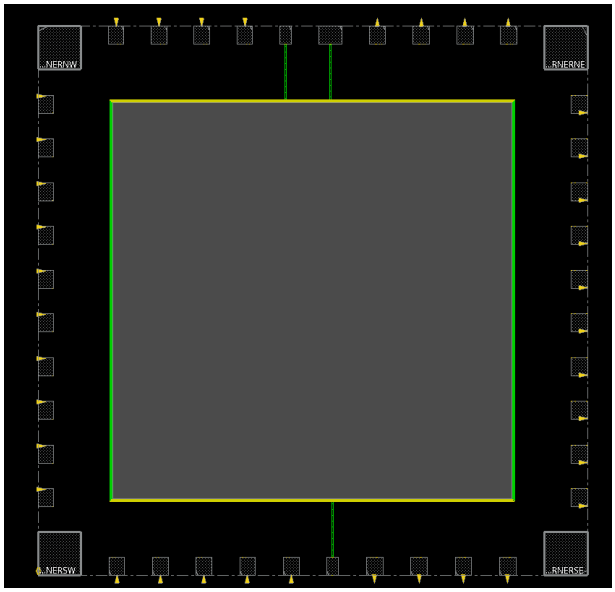
From GUI, click on "**Route**" and choose "**Special Route…**" to open "**Special Route**" window. Fill in the names of special nets and click "**OK**" or "**Apply**".



Special routing can also be achieved with the following commands:

```
>> setSrouteMode -viaConnectToShape { noshape }
>>  sroute  -connect  {  blockPin  padPin  padRing  corePin
floatingStripe  }  -layerChangeRange  {  MET1(1)  MET4(4)  }  -
blockPinTarget { nearestTarget } -padPinPortConnect { allPort
oneGeom } -padPinTarget { nearestTarget } -corePinTarget {
firstAfterRowEnd } -floatingStripeTarget { blockring padring
ring stripe ringpin blockpin followpin } -allowJogging 1 -
crossoverViaLayerRange { MET1(1) MET4(4) } -nets { gnd vdd } -
allowLayerChange 1 -blockPin useLef -targetViaLayerRange {
MET1(1) MET4(4) }
>> setLayerPreference node_layer -isVisible 0
>> setLayerPreference C1 -isVisible 1
>> setLayerPreference A1 -isVisible 1
>> setLayerPreference C2 -isVisible 1
>> selectWire 628.9500 1354.0000 633.0500 1354.1180 3 gnd
>> uiSetTool stretchWire
>>  editResize  -direction  y  -offset  -148.259  -side  low  -
keep_center_line auto
>> deselectAll
>> selectWire 742.9500 1354.0000 747.0500 1354.1180 3 vdd
>>  editResize  -direction  y  -offset  -148.384  -side  low  -
keep_center_line auto
>> editTrim
>> deselectAll
>> selectWire 748.9500 45.8820 753.0500 46.0000 3 gnd
>>  editResize  -direction  y  -offset  148.524  -side  high  -
keep_center_line auto
>> editTrim
>> deselectAll
```

The Innovus window generated is as shown below:



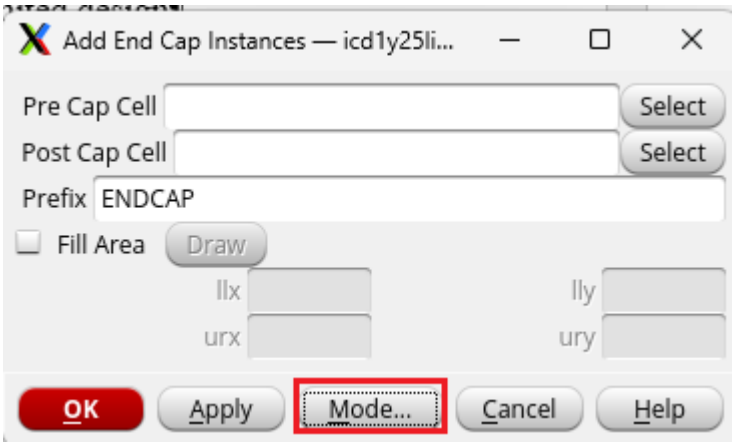
6. IO pad Filler Placement

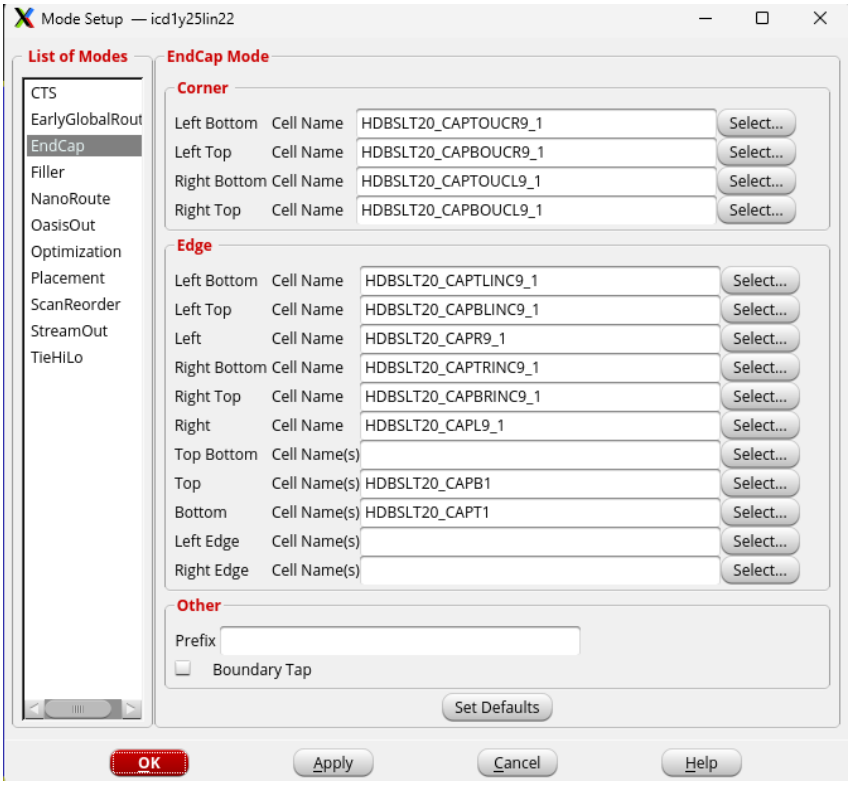
- **There should be no spacing between pads**
  - Therefore adding pad filler is necessary for core limited design
- PAD filler must be added from wider fillers to narrower ones, or there may have some DRC/LVS violations after PAD filler insertion

```
>> addIoFiller -cell "IN22FDX_GPIO18_10M19S40PI_FILL20_V
     IN22FDX_GPIO18_10M19S40PI_FILL10_V
     IN22FDX_GPIO18_10M19S40PI_FILL5_V
     IN22FDX_GPIO18_10M19S40PI_FILL1_V" -prefix IOFILLER_N -
     side n
>> addIoFiller -cell "IN22FDX_GPIO18_10M19S40PI_FILL20_V
     IN22FDX_GPIO18_10M19S40PI_FILL10_V
     IN22FDX_GPIO18_10M19S40PI_FILL5_V
     IN22FDX_GPIO18_10M19S40PI_FILL1_V" -prefix IOFILLER_S -
     side s
>> addIoFiller -cell "IN22FDX_GPIO18_10M19S40PI_FILL20_H
     IN22FDX_GPIO18_10M19S40PI_FILL10_H
     IN22FDX_GPIO18_10M19S40PI_FILL5_H
     IN22FDX_GPIO18_10M19S40PI_FILL1_H" -prefix IOFILLER_W -
     side w
>> addIoFiller -cell "IN22FDX_GPIO18_10M19S40PI_FILL20_H
     IN22FDX_GPIO18_10M19S40PI_FILL10_H
     IN22FDX_GPIO18_10M19S40PI_FILL5_H
     IN22FDX_GPIO18_10M19S40PI_FILL1_H" -prefix IOFILLER_E -
     side e
```

7. End cap Placement (subject to the requirement of the process/library)

End cap cells are needed to fulfil the well tie off specification and well extension requirement for DRC. To add them from GUI, click on "**Place**", select "**Physical Cell**" and choose "**Add End Cap…**". Choose appropriate End caps in "Mode" for placement and click "**OK**" or "**Apply**".



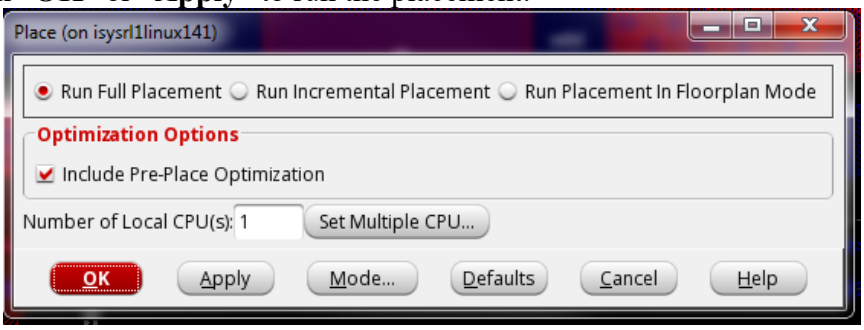


End cap cells can also be added by using the command below:

```
>> snapFPlan -all
>> set CELL_PREFIX "HDBSLT20"
>> setEndCapMode -reset
>>
>>
>> setEndCapMode -reset
>>  setEndCapMode \
    -topEdge "${CELL_PREFIX}_CAPB1" \
    -bottomEdge "${CELL_PREFIX}_CAPT1" \
    -leftEdge "${CELL_PREFIX}_CAPR9_1" \
    -rightEdge "${CELL_PREFIX}_CAPL9_1" \
    -rightTopCorner "${CELL_PREFIX}_CAPBOUCL9_1     " \
    -rightBottomCorner "${CELL_PREFIX}_CAPTOUCL9_1 " \
    -leftTopCorner "${CELL_PREFIX}_CAPBOUCR9_1      " \
    -leftBottomCorner "${CELL_PREFIX}_CAPTOUCR9_1   " \
    -leftBottomEdge "${CELL_PREFIX}_CAPTLINC9_1" \
    -lefttopEdge "${CELL_PREFIX}_CAPBLINC9_1" \
    -rightBottomEdge "${CELL_PREFIX}_CAPTRINC9_1" \
    -righttopEdge "${CELL_PREFIX}_CAPBRINC9_1" \
    -fitGap true
>> addEndCap -prefix ENDCAP
```

8. Standard Cell Placement

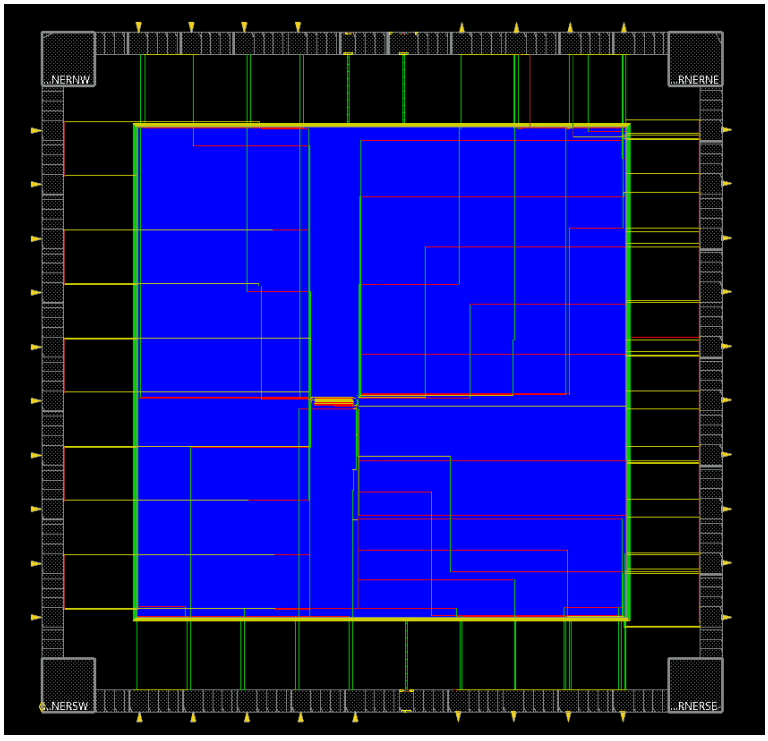
The following step involves placement of standard library cells used in the circuit. To place the cells from GUI, click on "**Place**" and select "**Place Standard Cell…**". On the "Place" window, click "**OK**" or "**Apply**" to run the placement.



The text commands for standard cell placement are:

```
>> setPlaceMode -fp false
>> placeDesign
```

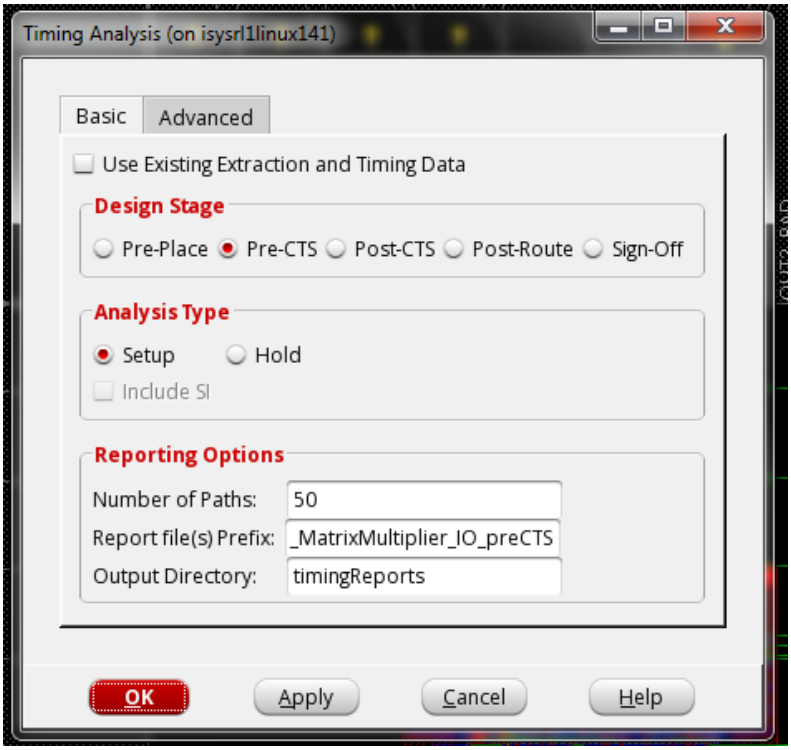
After running placement, the standard cells are placed in core region and design has been trial routed. The result as shown below:



9. Clock Tree Synthesis

In this part, Clock Tree Synthesis (CTS) and optimization will be done.

Before running CTS, timing analysis should be done to check if there is any setup time violation exists in any of the signal paths. To perform the check from GUI, click on "**Timing**", then select "**Report Timing…**". Check "**Pre-CTS**" under design stage and "**Setup**" Analysis Type and click "**OK**" or "**Apply**".



You will observe that a folder named "**timingReports"** will be generated within the folder Module 6. Select the timing Reports folder and choose the file named **Syn_MatrixMultiplier_IO_preCTS.summary**. You may visualise the contents of the file as similar to what is shown in the following window:

```
------------------------------------------------------------------
     optDesign Final Summary
------------------------------------------------------------------

Setup views included:
 typical rcworst_DP

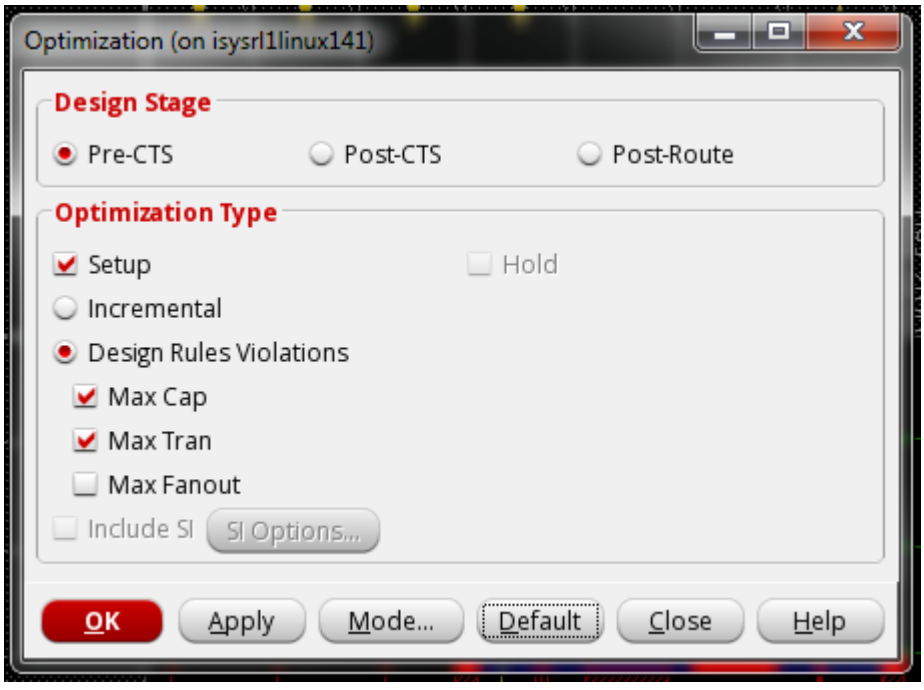
+--------------------+---------+---------+---------+
|     Setup mode     |   all   | reg2reg | default |
+--------------------+---------+---------+---------+
|           WNS (ns):| 90.034  | 96.646  | 90.034  |
|           TNS (ns):|  0.000  |  0.000  |  0.000  |
|    Violating Paths:|    0    |    0    |    0    |
|          All Paths:|   126   |   53    |   90    |
+--------------------+---------+---------+---------+

+----------------+-------------------------------+------------------+
|                |              Real             |       Total      |
|    DRVs        +------------------+------------+------------------|
|                |  Nr nets(terms)  | Worst Vio  |  Nr nets(terms)  |
+----------------+------------------+------------+------------------+
|   max_cap      |      0 (0)       |   0.000    |      0 (0)       |
|   max_tran     |      0 (0)       |   0.000    |      0 (0)       |
|   max_fanout   |      0 (0)       |     0      |      0 (0)       |
|   max_length   |      0 (0)       |     0      |      0 (0)       |
+----------------+------------------+------------+------------------+

Density: 0.023%
Routing Overflow: 0.00% H and 0.00% V
```

You will be able to see the table containing setup mode parameters. Violating paths is one of those parameters and in the above case no violating path exists as the number of violating paths is zero in all the mentioned cases. But if any setup time violating path exists, it should be optimized.

To optimise design from GUI, click on "**ECO**" and select "**Optimize Design…**". In "Optimization" window, select "**Pre-CTS**" under Design Stage and choose "**Setup**" under Optimization Type. If there are any design rules violations, choose "**Design Rules Violations**" and select appropriate types. If not, choose "**Incremental**".



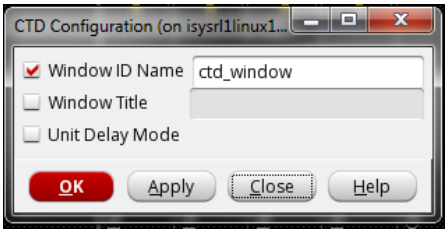
You can perform the "Hold" time analysis same way as "Setup" time analysis. However, note that there is no "Hold" time optimization for "Pre-CTS". It can only be optimized Post-CTS. Pre-CTS analysis and optimization can also be performed using the commands below:

```
>> timeDesign -preCTS -idealClock -pathReports -drvReports -
slackReports -numPaths 50 -prefix
Syn_MatrixMultiplier_IO_preCTS -outDir timingReports
>> setOptMode -fixCap true -fixTran true -fixFanoutLoad false
>> optDesign -preCTS
```

CTS can be performed after pre-CTS analysis and optimization. CTS can only be performed from text command and the commands are:

```
>> create_ccopt_clock_tree_spec
>> ccopt_design -prefix cts
```

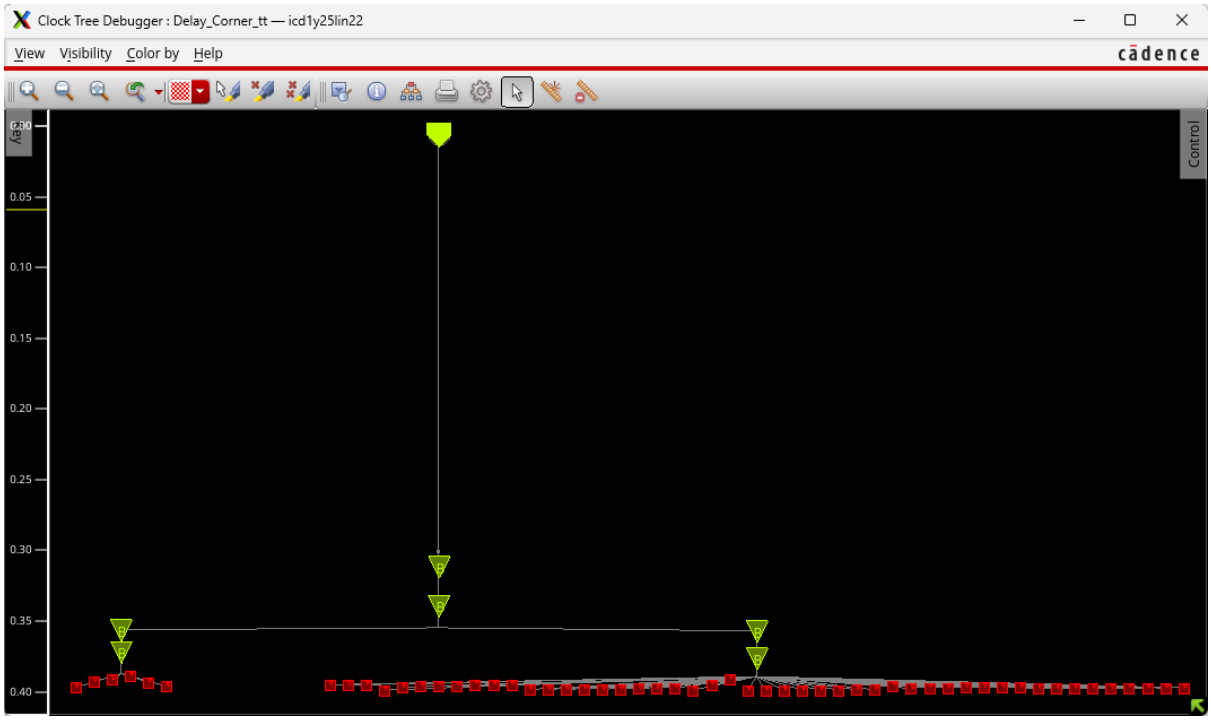
The clock tree can be view and trace from "Clock Tree Debugger" window. To open the window, click on "**Clock**" then select "**CCOpt Clock Tree Debugger**". By clicking "**OK**" on "CTD configuration" window, you will see the "Clock Tree Debugger" window.



Alternatively you can use the command below to open the "**Clock Tree Debugger**" window directly:

```
>> ctd_win -id ctd_window
```

The "**Clock Tree Debugger**" window is shown below. The information on clock tree such as clock root, buffer, buffer specification, connections and locations can be view, and clock tree can be trace from the window.



In a similar way, the setup time and hold time violation of signal paths after clock synthesis can be checked. The procedure is very similar to the check performed before clock tree synthesis.

The steps involved in checking and optimizing the setup time violation is similar to the steps mentioned above except instead of **pre-CTS** you have to choose the option **post-CTS** wherever required. Similarly the file which shows the violating paths information after clock synthesis is a file named **Syn_MatrixMultiplier_IO_postCTS.summary** generated within the timing Reports folder.

The steps involved in checking and optimizing the hold time violation is similar to the steps mentioned above except instead of **pre-CTS,** you have to choose the option **post-CTS** wherever required. Similarly the file which shows the violating paths information after clock synthesis is a file named **Syn_MatrixMultiplier_IO_postCTS_hold.summary** generated within the timing Reports folder.

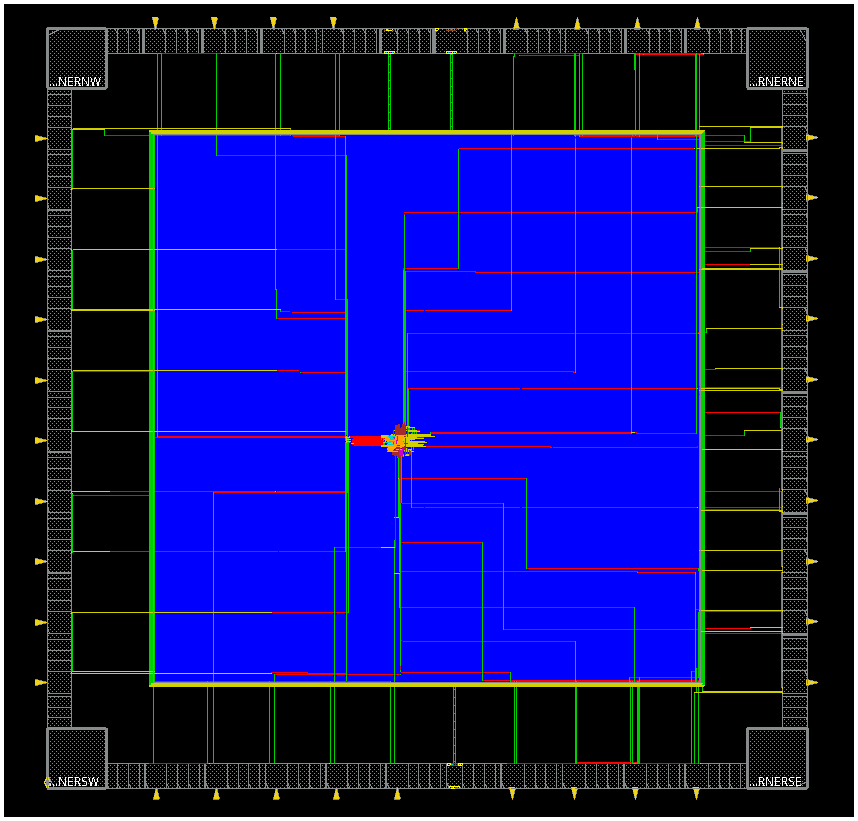
10. NanoRouting

The next major task required in the layout development is the formation of interconnections both within and between the different functional blocks present in the core circuit in order to realize the circuit functionality.

This task of routing also be performed by the following command:

```
>> source  ./scripts/Route.tcl
```

The Innovus window clearly displays the interconnection formed as given below:



Similar to pre-clock synthesis and post-clock synthesis, the setup time and hold time violations can be checked and optimized after the completion of routing. The analysis setting should be changed to on-chip variation to do post-Route analysis. To change from GUI, click on "**Tools**", select "**Set Mode**" and choose "**Specify Analysis Mode…**". On "Specify Analysis Mode…" window, choose "**On-Chip Variation**" under "**Timing Mode**".

It can also be done using text command as below:

```
>> setExtractRCMode -engine postRoute -effortLevel signoff -coupled true -capFilterMode relAndCoup -coupling_c_th 0.1 -total_c_th 0 -relative_c_th 1 -lefTechFileMap 22FDSOI_10M_2Mx_5Cx_1Jx_2Qx_LB_qrc_leftech.map
>> setAnalysisMode -cppr none -clockGatingCheck true -timeBorrowing true -useOutputPinCap true -sequentialConstProp false -timingSelfLoopsNoSkew false -enableMultipleDriveNet true -clkSrcPath true -warn true -usefulSkew true -analysisType onChipVariation -log true

>> redirect -quiet {set honorDomain [getAnalysisMode -honorClockDomains]} > /dev/null
>> timeDesign -signoff -pathReports -drvReports -slackReports -numPaths 50 -prefix CHIP_TOP_signOff -outDir timingReports
>> redirect -quiet {set honorDomain [getAnalysisMode -honorClockDomains]} > /dev/null
```

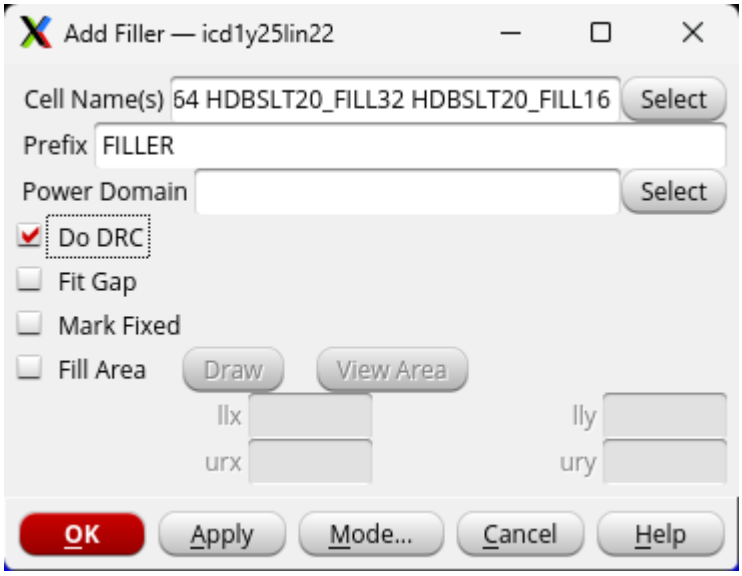
```
>> timeDesign -signoff -hold -pathReports -slackReports -
numPaths 50 -prefix CHIP_TOP_signOff -outDir timingReports
```

The steps involved in checking and optimizing the setup time violation is similar to the steps mentioned above except instead of **pre- CTS** or **post-CTS** you have to choose the option **post-Route** wherever required. Similarly the file which shows the violating paths information after routing is a file named **Syn_MatrixMultiplier_IO_postRoute.summary** generated within the timing Reports folder.

The steps involved in checking and optimizing the hold time violation is similar to the steps mentioned above except instead of **pre-CTS** or **post-CTS**, you have to choose the option **post-Route** wherever required. Similarly the file which shows the violating paths information after routing is a file named **Syn_MatrixMultiplier_IO_postRoute_hold.summary** generated within the timing Reports folder.

11. Filler Cells Placement

The next step involved in completing the layout is the addition of core filler cells. The filler cells are required for continuity of wells and to avoid design rule violation. To add them from GUI, click on "**Place**", select "**Physical Call**" then choose "**Add Filler…**". Specify filler cells names and check "**Do DRC**" on "**Add Filler**" window.

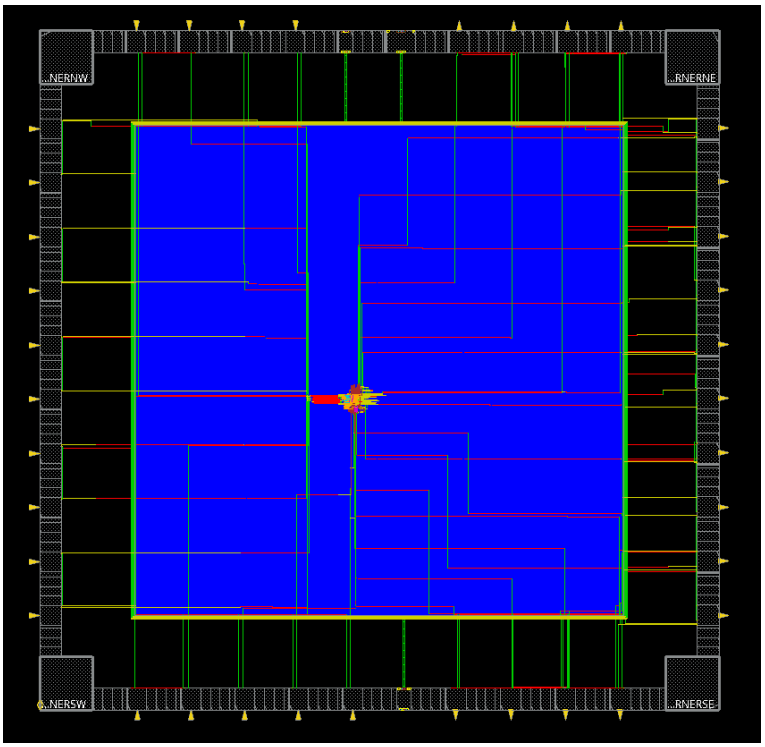


This can also be achieved by the following command:

```
>> getFillerMode -quiet
>>    addFiller    -cell    HDBSLT20_FILL64    HDBSLT20_FILL32
HDBSLT20_FILL16  HDBSLT20_FILL8  HDBSLT20_FILL4  HDBSLT20_FILL3
HDBSLT20_FILL2 HDBSLT20_FILL1 -prefix FILLER -doDRC
```

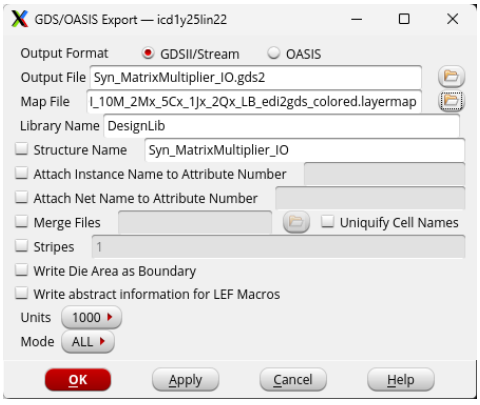
```
>> source  ./scripts/Route.tcl
```

The Innovus window is thus modified as shown (note that wires are hidden):



12. GDS2 Extraction

Thus the complete layout of the matrix multiplier circuit is successfully placed and routed. To export out the layout (in GDS format) from GUI, click on "**File**", select "**Save**" and choose "**GDS/OASIS…**". On "GDS/OASIS Export" window, specify the output file name and Map file (provided by foundry), and click "**OK**".



You can also extract the layout in GDS format using command as follows:

```
>> streamOut Syn_MatrixMultiplier_IO.gds2 -mapFile
GDS/22FDSOI_10M_2Mx_5Cx_1Jx_2Qx_LB_edi2gds_colored.layermap -
libName DesignLib -structureName Syn_MatrixMultiplier_IO -
units 1000 -mode ALL
```

13. Parasitic Extraction

The parasitic information from the layout can be extracted for post-layout simulation and verification. To extract from GUI, click "**Timing**", and select "**Extract RC…**". On "Extract RC" window, check the parasitic file format that you need (in this case .spef) and click "**OK**".

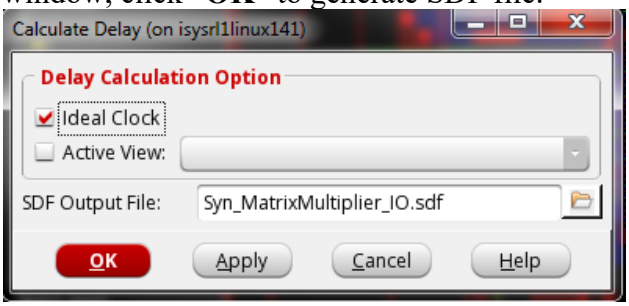


The parasitic extraction can be done by the following commands:

```
>> rcOut -spef Syn_MatrixMultiplier_IO.spef -rc_corner typical
```

14. Timing File Generation

The timing file/ SDF (Standard Delay Format) file can also be generated for post-layout simulation and verification. To generate from GUI, click "**Timing**" and select "**Write SDF…**". On "Calculate Delay" window, click "**OK**" to generate SDF file.

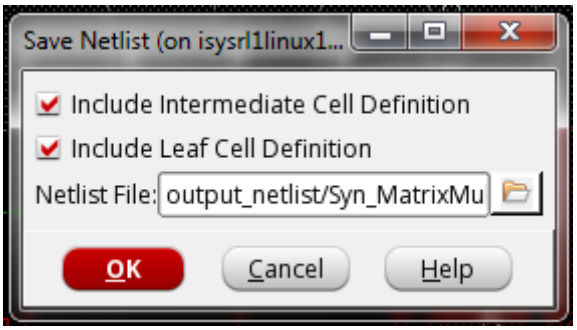


It can also be done using the command as follows:

```
>> all_hold_analysis_views
>> all_setup_analysis_views
>> write_sdf Syn_MatrixMultiplier_IO.sdf
```

15. Post-Layout Netlist Extraction

The routed Verilog netlist is required to do post-layout verification and simulation. To generate it from GUI, click on "**File**", select "**Save**" and choose "**Netlist…**". On "Save Netlist" window, click "**OK**" to save the Verilog netlist.



The Verilog file can also be extracted using the command below:

```
>> saveNetlist output_netlist/Syn_MatrixMultiplier_IO.v
```